

# Threshold Fully Deniable Interactive Encryption <sup>★</sup>

Anonymous

**Abstract.** Deniable Encryption (Canetti et al., CRYPTO 1997) enables parties to produce fake internal states making any transcript consistent with any plaintext of their choice. Fully Deniable Encryption (FDE) (Canetti et al., CRYPTO 2020) achieves the strongest two-party guarantee: both sender and receiver can independently fabricate randomness, even with mutually inconsistent claimed plaintexts, and anyone can fake the receiver’s side. However, FDE addresses only post-execution coercion and remains confined to the two-party setting, which leaves both *adaptive coercion* (where parties may be coerced at any round during execution) and *threshold deniability* (where secrets are distributed among  $n$  parties) as open problems.

In this work, we resolve both gaps; we design *Threshold Fully Deniable Interactive Encryption* (TFDE), composing FDE with a universal thresholdizer, equivocal commitments, and non-committing encryption via a four-layer equivocation pipeline so that any coalition of fewer than  $t_S$  senders and  $t_R$  receivers can produce per-party fake states indistinguishable from honest executions, while satisfying all standard threshold public-key encryption properties. Then, we present *Strong Off-the-Record Deniability*, which captures adaptive mid-execution coercion and adds a new indistinguishability case for consistent fake claims by both parties.

**Keywords:** Cryptographic Distributed Protocols, Secure Multi-party Computation, Privacy-preserving Protocols, Fully Deniable Encryption

## 1 Introduction

The ability to communicate secretly over public channels without any previously shared secrets is, indeed, one of the fundamental principles of contemporary cryptography [1]. Nonetheless, traditional concepts of secure communication provide secrecy only if the internal states of the parties remain concealed [2], and it has been proven that non-interactive non-committing encryption is impossible [3]. In case either the sender or the receiver is later forced or given money to disclose their secret keys and randomness, conventional encryption methods will not protect them [4]. Additionally, in multiple protocols, the communication log practically binds the parties to the disclosed message, thus creating a situation where asserting that another message was sent becomes non-credible.

*Deniable Encryption.* Introduced by Canetti et al [5], Deniable Encryption addresses this limitation by enabling parties to produce fake internal states that make any transcript appear consistent with any plaintext of their choice [5,6]. Deniable encryption allows user to decrypt the same ciphertext to different plausible plaintexts depending on which secret user uses, so user can reveal a "decoy" message under coercion while hiding the real one [6]. Since its emergence, deniable encryption has shown promising real-world applications such as Smartphones [7], Signal Messenger [8], IoT and wearable

---

<sup>★</sup> This submission is eligible for the Best Student Regular paper award.

devices [9], MobiWear for smartwatches [10], FractalCrypt [11], and VeraCrypt [12, 13]. The defining feature of deniable encryption is that an adversary cannot distinguish whether the parties revealed their true randomness and plaintext or provided fabricated alternatives [5, 6]. This guarantee holds even when the adversary observes the entire communication and later coerces the parties.

*Fully Deniable Encryption.* The development of deniable encryption has proceeded incrementally through a series of partial results: in sender-deniable protocol [14] only sender can produce fake randomness; in receiver-deniable protocol [14, 15], only receiver can produce fake randomness; in bideniable protocol [15], both sender and receiver can fake simultaneously; in Fully Deniable Encryption (FDE) [16], both sender and receiver can fake their randomness after execution, even when their claimed plaintexts are mutually inconsistent, and anyone can fake the receiver’s side.

*Problem 1: Stronger Deniability.* The construction of Canetti et al. [16] addresses only post-execution coercion, where adversaries coerce parties after the protocol completes. The more challenging setting of *adaptive coercion*, where parties may be coerced at any point during protocol execution, was identified in [16] as an open problem.

*Threshold Cryptosystems.* The more technologies branches from distributed systems [17] to LLMs [18] and federated learning [19], the need for distributed/multi-party threshold security in privacy-preserving protocols is becoming more highlighted, where NIST recently announced the call for standardization of Threshold Cryptography [20, 21]. Threshold cryptosystem distributes a secret across  $N$  parties such that any subset of at least  $t$  parties can collectively reconstruct or operate on the secret [22], while fewer than  $t$  parties gain zero information about it; a Threshold Public-Key Encryption (TPKE) scheme is where decryption requires cooperation of at least  $t$  out of  $n$  keyholders, with no single party holding the full secret key [23]. Real-world applications of distributed security as threshold cryptosystems can be considered from Microsoft ElectionGuard [24, 25] and AWS CloudHSM [26] to DigiCert [27], and PySyft [28] of OpenMined.

*Problem 2: Threshold Fully Deniable Protocol.* Putting deniable encryption and importance of threshold cryptosystems together, the point that highlights is that there is no Threshold Fully Deniable Interactive Encryption (TFDE) protocol to this date. So, as Figure 1 demonstrates, the question is:

*Given a two-party FDE, can one construct a TFDE in which any coalition of fewer than  $t_S$  senders and fewer than  $t_R$  receivers can jointly encrypt, decrypt, and produce fake internal states that are indistinguishable from honest executions under any plaintext?*

This is not trivial because of multiple challenges: (I) existing FDE faking [16] operates on monolithic randomness, not individual shares, (II) a single fake randomness  $s'$  does not produce equivocal shares, partial evaluations, commitments, and channel randomness at the individual-party level, (III) simultaneously coerced parties across both sides require mutually consistent fake views.

## 1.1 Contributions

Initially, we resolve *Problem 2* by introducing *Threshold Fully Deniable Interactive Encryption* (TFDE, Definition 5), a multi-party protocol in which any coalition of fewer

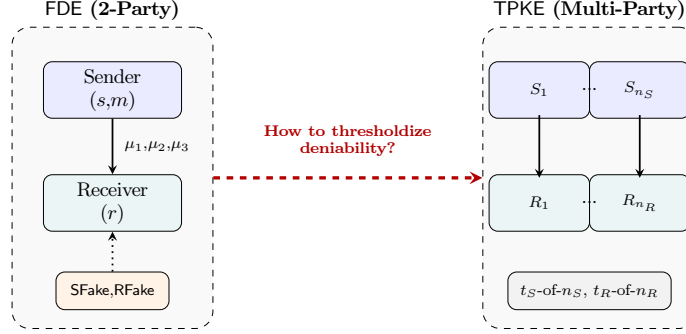


Fig. 1: The TFDE problem, where FDE is with sender randomness  $s$ , receiver randomness  $r$ , plaintext  $m \in \mathcal{M}$ , transcript  $(\mu_1, \mu_2, \mu_3)$ , and faking algorithms  $\text{SFake}$ ,  $\text{RFake}$ ; TPKE is over  $n_S$  sender parties  $S_1, \dots, S_{n_S}$  with threshold  $t_S$  and  $n_R$  receiver parties  $R_1, \dots, R_{n_R}$  with threshold  $t_R$ .

than  $t_S$  senders and  $t_R$  receivers can produce per-party fake internal states, indistinguishable from honest executions under any plaintext. Our construction composes FDE [16], a universal thresholdizer [23, 29–31], equivocal commitments [32], and non-committing encryption [33] via a four-layer equivocation pipeline, which satisfies threshold bideniability, off-the-record deniability, and public receiver deniability (Theorems 1–4), and is secure against accumulating attacks [34]. Eventually, we address *Problem 1* by introducing *Strong Off-the-Record Deniability* (SORD, Definition 11), which strengthens off-the-record deniability [16] along two aspects: (i) it captures *adaptive coercion*, where parties may be coerced at any round during execution, and (ii) it adds a fourth indistinguishability case covering both parties deviating with *consistent* fake claims.

## 1.2 Related Works

As Figure 2 demonstrates, a summary of the evolution direction of protocols, the concept of deniable encryption was introduced by Canetti, Dwork, Naor, and Ostrovsky in their work [5], which established the foundational definitions and provided initial constructions. However, their original scheme achieved only an inverse polynomial distinguishing advantage, requiring superpolynomially long ciphertexts to obtain security. This limitation persisted for nearly two decades, during which the community sought more efficient constructions and explored the boundaries of what could be achieved.

Sahai and Waters [14] achieved the first sender-deniable encryption with polynomial-length ciphertexts via indistinguishability obfuscation, transforming randomized algorithms into deniable versions through obfuscated programs generating fake randomness. Their scheme converts to receiver-deniable encryption via the generic transformation from [5], yielding a three-message protocol handling adversaries coercing either party separately, but not both simultaneously.

Impossibility results shaped understanding of deniable encryption’s limitations. Bendlin, Nielsen, Nordholt, and Orlandi [34] proved two-message receiver-deniable encryption impossible via information-theoretic compressibility arguments, establishing three-round necessity for bideniable schemes. Nielsen showed non-committing encryption permits only bounded public key reuse, implying deniable communication must

be interactive. Dachman-Soled [2] established black-box construction limitations for sender-deniable encryption from simulatable public-key encryption.

O’Neill, Peikert, and Waters [15] introduced multi-distributional bideniability under restricted settings, though their construction does not achieve standard bideniability where adversaries know the scheme and see all random coins. Unrestricted bideniable encryption remained open until Canetti, Park, and Poburinnaya [16] work.

Canetti, Park, and Poburinnaya [16] resolved the bideniable encryption existence question, where they presented a three-message protocol achieving bideniability and off-the-record deniability (protecting parties with inconsistent claimed states). Their construction requires subexponentially secure indistinguishability obfuscation and one-way functions, operates in the CRS model, addresses post-execution coercion, and explicitly identifies adaptive coercion and weaker assumption constructions as open problems.

Extensions to richer functionalities include: Apon, Fan, and Liu [4] constructed deniable attribute-based encryption from LWE for branching programs; De Caro, Iovino, and O’Neill [6] introduced deniable functional encryption denying both plaintexts and computed functions; Goldwasser, Klein, and Wichs [1] studied deniable editing of encrypted data with plausible deniability-preserving ciphertext modifications.

Of course, deniable security is not limited to just encryption schemes: Jiang et al. [35] proposed a formal framework for deniable authenticated key exchange to make sure both session key indistinguishability and offline deniability against any computationally bounded adversary; also, Canetti et al. [36] proposed a secret sharing scheme that is deniable, which allows shareholders to produce fake-but-plausible shares that open to any chosen secret, thus concealing participation under coercion.

Nevertheless, no attempt has been done to achieve threshold encryption structures with deniable properties.

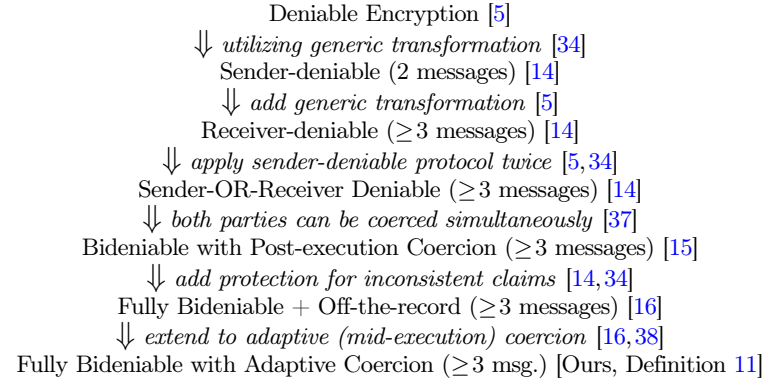


Fig. 2: The hierarchy of deniability by the number of compromised parties.

## 2 Preliminaries

We consider *access structures* as all sets of authorized parties  $A \subset \mathbb{S}$  with  $N$  total parties  $|\mathbb{A}| \leq N$ . Access structures are *efficient* if they are monotone (i.e.,  $\forall A \subset \mathbb{S}$

and  $|\mathbb{A}| < N$  implies  $\mathbb{A} \cup \{x\} \subset \mathbb{S}$ ) and they can be described using a boolean circuit  $f_{\mathbb{A}} : \{0,1\}^N \rightarrow \{0,1\}$ . We use circuits of arbitrary inputs  $\mathbb{C} : \{0,1\}^k \rightarrow \{0,1\}$  that are bounded by depth  $d$ . For simplicity in some parts, we use  $C \in \mathbb{C}$ .

**Definition 1 (Equivocable Commitment).** An equivocable commitment scheme over a domain  $\mathcal{X} = \{\mathcal{X}_\lambda\}_\lambda \in \mathbb{N}$  is a 5-tuple  $\text{EQ} = (\text{EQ.Setup}, \text{EQ.Commit}, \text{EQ.Decommit}, \text{EQ.Equiv}_1, \text{EQ.Equiv}_2)$  of PPT algorithms:

- $\text{crs} \leftarrow \text{EQ.Setup}(1^\lambda)$ : Takes security parameter; outputs a common reference string  $\text{crs}$ .
- $(c, d) \leftarrow \text{EQ.Commit}(1^\lambda, \text{crs}, x)$ : Takes security parameter  $1^\lambda$ , a common reference string  $\text{crs}$ , and an element  $x \in \mathcal{X}_\lambda$ ; outputs a commitment  $c$  and a decommitment  $d$ .
- $x/\perp \leftarrow \text{EQ.Decommit}(1^\lambda, \text{crs}, c, d)$ : Takes security parameter  $1^\lambda$ , a common reference string  $\text{crs}$ , a commitment  $c$ , and a decommitment  $d$ ; outputs an element  $x \in \mathcal{X}_\lambda$  or the rejection symbol  $\perp$ .
- $(\widetilde{\text{crs}}, \hat{c}, \text{st}) \leftarrow \text{EQ.Equiv}_1(1^\lambda)$ : Takes security parameter  $1^\lambda$ ; outputs a simulated common reference string  $\widetilde{\text{crs}}$ , a simulated commitment  $\hat{c}$ , and a state  $\text{st}$ .
- $\hat{d} \leftarrow \text{EQ.Equiv}_2(1^\lambda, \text{st}, x)$ : Takes security parameter  $1^\lambda$ , a state  $\text{st}$ , and an element  $x \in \mathcal{X}_\lambda$ ; outputs a decommitment  $\hat{d}$  that opens  $\hat{c}$  to  $x$ .

Based on [32], EQ satisfies three properties: Correctness if for any  $x \in \mathcal{X}_\lambda$ , running  $\text{crs} \leftarrow \text{EQ.Setup}(1^\lambda)$  and  $(c, d) \leftarrow \text{EQ.Commit}(1^\lambda, \text{crs}, x)$  yields  $\text{EQ.Decommit}(1^\lambda, \text{crs}, c, d) = x$  with probability 1; Equivocability: if there exist algorithms  $\text{EQ.Equiv}_1$  and  $\text{EQ.Equiv}_2$  such that  $(\widetilde{\text{crs}}, \hat{c}, \hat{d})$  produced by first running  $(\widetilde{\text{crs}}, \hat{c}, \text{st}) \leftarrow \text{EQ.Equiv}_1(1^\lambda)$  and then  $\hat{d} \leftarrow \text{EQ.Equiv}_2(1^\lambda, \text{st}, x)$  is indistinguishable from an honest  $(\text{crs}, c, d)$  for any  $x$  chosen by a PPT adversary, while still satisfying  $\text{EQ.Decommit}(1^\lambda, \widetilde{\text{crs}}, \hat{c}, \hat{d}) = x$ . Binding: if no adversary, given  $\text{crs} \leftarrow \text{EQ.Setup}(1^\lambda)$ , can produce  $(c, d, d', x, x')$  with  $x \neq x'$  s.t.  $\text{EQ.Decommit}(1^\lambda, \text{crs}, c, d) = x$  and  $\text{EQ.Decommit}(1^\lambda, \text{crs}, c, d') = x'$ , except with negligible probability.

**Definition 2 (Non-Committing Encryption).** A non-committing encryption (NCE) scheme is a tuple of PPT algorithms  $\text{NCE} = (\text{Gen}, \text{Enc}, \text{Dec}, \text{Sim}_1, \text{Sim}_2)$ :

- $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda, r_G)$ : Takes security parameter  $1^\lambda$  and randomness  $r_G$ ; outputs a public key  $\text{pk}$  and a secret key  $\text{sk}$ .
- $\text{ct} \leftarrow \text{Enc}(\text{pk}, M, r_E)$ : Takes public key, message, randomness  $r_E$ ; outputs ciphertext  $\text{ct}$ .
- $M/\perp \leftarrow \text{Dec}(\text{sk}, \text{ct})$ : Takes a secret key  $\text{sk}$  and a ciphertext  $\text{ct}$ ; outputs a message  $M$  or the rejection symbol  $\perp$ .
- $(\text{pk}, \text{ct}, \text{st}) \leftarrow \text{Sim}_1(1^\lambda)$ : Takes security parameter  $1^\lambda$ ; outputs a simulated public key  $\text{pk}$ , a simulated ciphertext  $\text{ct}$ , and an internal state  $\text{st}$ .
- $(r_G, r_E) \leftarrow \text{Sim}_2(M, \text{st})$ : Takes a message  $M$  and an internal state  $\text{st}$ ; outputs randomness  $r_G$  for key generation and randomness  $r_E$  for encryption, equivocating the simulated  $(\text{pk}, \text{ct})$  to  $M$ .

Considering [33] and [37], an NCE for Correctness requires that the probability  $\Pr[\text{Dec}(\text{sk}, \text{Enc}(\text{pk}, M)) = M] \geq 1 - \text{negl}(\lambda)$ . For Simulatability requires that the real distribution  $\{(\text{pk}, c, r_G, r_E) : (\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda; r_G), c \leftarrow \text{Enc}(\text{pk}, M; r_E)\}$  is computationally indistinguishable from the ideal distribution  $\{(\text{pk}, c, r_G, r_E) : (\text{pk}, c, \text{st}) \leftarrow \text{Sim}_1(1^\lambda), (r_G, r_E) \leftarrow \text{Sim}_2(M, \text{st})\}$ , where in both worlds  $M \leftarrow \mathcal{A}(\text{pk})$  for any PPT adversary  $\mathcal{A}$ .

**Definition 3 (Universal Thresholdizer).** Consider the efficient access structure  $(\mathbb{S}, N)$  and circuit class  $(\mathbb{C}, d)$ . A universal thresholdizer  $\text{UT}$  consists of the algorithms:

- $(\text{pp}, s_1, \dots, s_N) \leftarrow \text{Setup}(1^\lambda, 1^d, (\mathbb{S}, N), x)$ : Given the security parameter  $1^\lambda$ , a depth bound  $1^d$ , an efficient access structure  $(\mathbb{S}, N)$ , and a message  $x \in \{0, 1\}^k$ , this algorithm outputs public parameters  $\text{pp}$  and a set of shares  $s_1, \dots, s_N$ .
- $y_i \leftarrow \text{Eval}(\text{pp}, s_i, \mathbb{C})$ : On input of the public parameters  $\text{pp}$ , a share  $s_i$ , and a circuit  $\mathbb{C}: \{0, 1\}^k \rightarrow \{0, 1\}$  of depth at most  $d$ , the algorithm produces a partial evaluation  $y_i$ .
- $\{0, 1\} \leftarrow \text{Verify}(\text{pp}, y_i, \mathbb{C})$ : This algorithm takes as input  $\text{pp}$ , a purported partial evaluation  $y_i$ , and a circuit  $\mathbb{C}$ , and outputs a bit indicating whether the evaluation is valid.
- $y \leftarrow \text{Combine}(\text{pp}, B)$ : Outputs the final evaluation  $y$ , given the collection of partial evaluations  $B = \{y_i\}_{i \in S}$  from an authorized set  $S$ .

According to [31], a UT satisfies the following properties. Compactness: the size of each partial evaluation  $y_i \leftarrow \text{UT.Eval}(\text{pp}, s_i, C)$  is bounded by  $\text{poly}(\lambda, d, N)$ , independent of the secret  $x$ . Evaluation correctness: for any authorized set of  $S \in \mathcal{A}$ ,  $\text{UT.Combine}(\text{pp}, \{\text{UT.Eval}(\text{pp}, s_i, C)\}_{i \in S}) = C(x)$  with overwhelming probability. The Verification correctness: for any honestly computed  $y_i$ ,  $\text{UT.Verify}(\text{pp}, y_i, C) = 1$  with overwhelming probability. Security: the shares  $\{s_i\}_{i \in S^*}$  and evaluation shares  $\{y_i\}_{i \in S}$  for any unauthorized  $S^* \notin \mathcal{A}$  are simulatable given only  $C$  and  $C(x)$ , i.e., no PPT adversary can distinguish the real experiment from an ideal one in which partial evaluations are replaced by a simulator  $\mathcal{S}(\text{pp}, C, C(x), S)$ . Robustness: no PPT adversary, given all shares  $s_1, \dots, s_N$ , can produce a fake partial evaluation  $y_i^* \neq \text{UT.Eval}(\text{pp}, s_i, C)$  that passes  $\text{UT.Verify}$ , except with negligible probability.

**Definition 4 (Fully Deniable Interactive Encryption).** A 3-message fully deniable interactive encryption (FDE) scheme for message space  $\mathcal{M}$  in the CRS model is a tuple of PPT algorithms  $\text{FDE} = (\text{Setup}, P_1, P_2, P_3, \text{Dec}, \text{SFake}, \text{RFake})$ :

- $\text{CRS} \leftarrow \text{Setup}(1^\lambda)$ : Takes security parameter  $1^\lambda$ ; outputs a common reference string  $\text{CRS}$  containing the descriptions of the six algorithms below.
- $\mu_1 \leftarrow P_1(s, m)$ : Takes sender randomness  $s$  and plaintext  $m \in \mathcal{M}$ ; outputs the first protocol message  $\mu_1$ .
- $\mu_2 \leftarrow P_2(r, \mu_1)$ : Takes receiver randomness  $r$  and first message  $\mu_1$ ; outputs the second protocol message  $\mu_2$ .
- $\mu_3 \leftarrow P_3(s, m, \mu_1, \mu_2)$ : Takes sender randomness  $s$ , plaintext  $m$ , and messages  $\mu_1, \mu_2$ ; outputs the third protocol message  $\mu_3$ .
- $m/\perp \leftarrow \text{Dec}(r, \mu_1, \mu_2, \mu_3)$ : Takes receiver randomness  $r$  and transcript  $(\mu_1, \mu_2, \mu_3)$ ; outputs plaintext  $m \in \mathcal{M}$  or  $\perp$ .
- $s' \leftarrow \text{SFake}(s, m, \hat{m}, \mu_1, \mu_2, \mu_3)$ : Takes sender randomness  $s$ , real plaintext  $m$ , fake plaintext  $\hat{m} \in \mathcal{M}$ , and transcript  $(\mu_1, \mu_2, \mu_3)$ ; outputs fake sender randomness  $s'$  that makes the transcript look consistent with  $\hat{m}$ .
- $r' \leftarrow \text{RFake}(\hat{m}, \mu_1, \mu_2, \mu_3; \rho)$ : Takes fake plaintext  $\hat{m} \in \mathcal{M}$ , transcript  $(\mu_1, \mu_2, \mu_3)$ , and fresh random coins  $\rho$ ; outputs fake receiver randomness  $r'$  that makes the transcript look consistent with  $\hat{m}$ . Note that  $\text{RFake}$  does not require the real receiver randomness  $r$  or the real plaintext  $m$  (public receiver deniability).

As formalized in [16], a FDE has multiple properties: Correctness requires that  $\text{Dec}(r, \text{tr}(s, r, m)) = m$  except with negligible probability over uniform  $s, r$ . Bideniability

requires that no PPT adversary can distinguish a real execution  $(s^*, r^*, \text{tr}^*)$  for plaintext  $m_0$  from a faked execution  $(s', r', \text{tr}^*)$  for plaintext  $m_1$ , where  $\text{tr}^* = \text{tr}(s^*, r^*, m_1)$ ,  $s' \leftarrow \text{SFake}(s^*, m_1, m_0, \text{tr}^*; \rho_S)$ , and  $r' \leftarrow \text{RFake}(r^*, m_1, m_0, \text{tr}^*; \rho_R)$ ; i.e.,  $(\text{tr}(s, r, m_0), s, r) \approx_c (\text{tr}(s, r, m_1), s', r')$ . Off-the-record deniability strengthens this by requiring that no PPT adversary can distinguish among three cases: (i) sender tells the truth and receiver fakes  $((s', r', \text{tr}^*)$  with  $\text{tr}^* = \text{tr}(s^*, r^*, m_0)$ ,  $r' \leftarrow \text{RFake}(r^*, m_0, m_1, \text{tr}^*)$ ); (ii) sender fakes and receiver tells the truth  $((s', r', \text{tr}^*)$  with  $\text{tr}^* = \text{tr}(s^*, r^*, m_1)$ ,  $s' \leftarrow \text{SFake}(s^*, m_1, m_0, \text{tr}^*)$ ); and (iii) both fake  $((s', r', \text{tr}^*)$  with  $\text{tr}^* = \text{tr}(s^*, r^*, m_2)$  for any  $m_2 \in \mathcal{M}$ ), ensuring protection even when parties' claimed plaintexts are mutually inconsistent. Finally, Public Receiver Deniability holds if  $\text{RFake}$  does not require the true receiver randomness  $r^*$  or true plaintext as input, i.e., it has syntax  $\text{RFake}(m', \text{tr}; \rho_R)$ , meaning any party (not just the receiver) can produce valid fake receiver coins.

### 3 Thresholdizing Deniable Encryption

We now present our construction, which lifts the two-party FDE protocol to a multi-party threshold setting by composing it with a universal thresholdizer, equivocable commitments, and non-committing encryption as demonstrated in Figure 3. Let  $\text{FDE} = (\text{FDE.Setup}, P_1, P_2, P_3, \text{FDE.Dec}, \text{SFake}, \text{RFake})$  be a 3-message fully deniable interactive encryption scheme (Definition 4), let  $\text{UT} = (\text{UT.Setup}, \text{UT.Eval}, \text{UT.Verify}, \text{UT.Combine})$  be a universal thresholdizer (Definition 3), let  $\text{EQ} = (\text{EQ.Setup}, \text{EQ.Commit}, \text{EQ.Decommit}, \text{EQ.Equiv}_1, \text{EQ.Equiv}_2)$  be an equivocable commitment scheme (Definition 1), and  $\text{NCE} = (\text{NCE.Gen}, \text{NCE.Enc}, \text{NCE.Dec}, \text{NCE.Sim}_1, \text{NCE.Sim}_2)$  be a non-committing encryption scheme (Definition 2).

**Definition 5 (Threshold Fully Deniable Interactive Encryption TFDE).** A Threshold Fully Deniable Interactive Encryption scheme for message space  $\mathcal{M}$ , sender access structure  $(\mathbb{S}_S, n_S)$  with threshold  $t_S$ , and receiver access structure  $(\mathbb{S}_R, n_R)$  with threshold  $t_R$ , in the CRS model, is a tuple of PPT algorithms  $\text{TFDE} = (\text{Setup}, \text{ShareGen}, \text{ThreshP}_1, \text{ThreshP}_2, \text{ThreshP}_3, \text{ThreshDec}, \text{ThreshSFake}, \text{ThreshRFake})$  defined as Figures 4, 5, 6, and 7.

*Setup.* The protocol initializes by  $\text{CRS}_{\text{FDE}} \leftarrow \text{FDE.Setup}(1^\lambda)$ ,  $\text{crs}_{\text{EQ}} \leftarrow \text{EQ.Setup}(1^\lambda)$ , and generating an NCE key pair  $(\text{pk}_{i,j}, \text{sk}_{i,j}) \leftarrow \text{NCE.Gen}(1^\lambda, r_{i,j}^G)$  for every ordered pair  $(i, j)$  spanning both intra-side pairs  $(i, j \in [n_S] \text{ or } i, j \in [n_R], i \neq j)$  and cross-side pairs  $(i \in [n_S], j \in [n_R])$ . The compound CRS is  $\text{CRS}_{\text{TFDE}} = (\text{CRS}_{\text{FDE}}, \text{crs}_{\text{EQ}}, \{\text{pk}_{i,j}, \text{sk}_{i,j}\}_{i,j})$ .

*Share Generation.* Given plaintext  $m$ , sender master randomness  $s$ , and receiver master randomness  $r$ , the dealer forms the sender secret  $x_S = (s \| m)$  and runs  $(\text{pp}_S, s_1^S, \dots, s_{n_S}^S) \leftarrow \text{UT.Setup}(1^\lambda, 1^d, (\mathbb{S}_S, n_S), x_S)$ , and analogously forms  $x_R = r$  and runs  $(\text{pp}_R, s_1^R, \dots, s_{n_R}^R) \leftarrow \text{UT.Setup}(1^\lambda, 1^d, (\mathbb{S}_R, n_R), x_R)$ . Each share  $s_i^S$  (resp.  $s_j^R$ ) is delivered to its holder via an NCE channel, to make sure all transmitted material is non-committingly encrypted and hence equivocable from the outset.



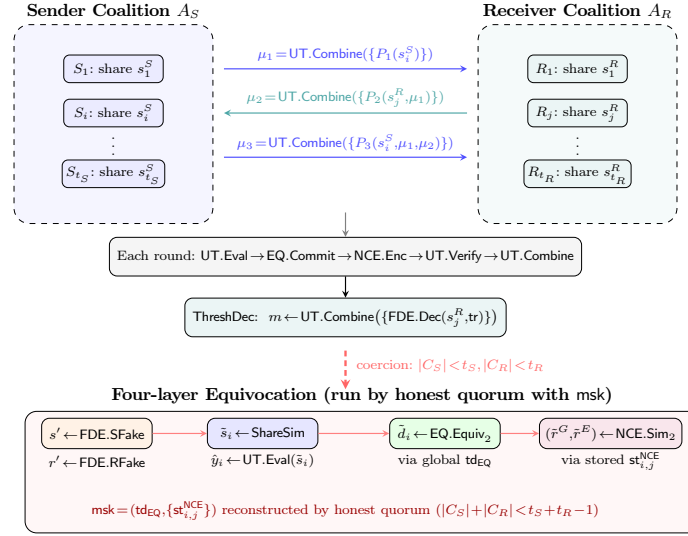


Fig. 3: Overview of our TFDE construction. The sender coalition ( $t_S$ -of- $n_S$ ) and receiver coalition ( $t_R$ -of- $n_R$ ) execute three interactive rounds, each following the pattern  $\text{UT.Eval} \rightarrow \text{EQ.Commit} \rightarrow \text{NCE.Enc} \rightarrow \text{UT.Verify} \rightarrow \text{UT.Combine}$ . Under coercion of  $|C_S| < t_S$  senders and  $|C_R| < t_R$  receivers, the honest quorum reconstructs  $\text{msk}$  and activates the four-layer equivocation pipeline (bottom): (1) FDE-level faking produces fake master randomness ( $s', r'$ ); (2)  $\text{ShareSim}$  re-explains coerced shares; (3)  $\text{EQ.Equiv}_2$  equivocates commitments; (4)  $\text{NCE.Sim}_2$  equivocates channel ciphertexts.

| TFDE.Setup( $1^\lambda$ )                                                                                                                | TFDE.ShareGen( $\text{CRS}_{\text{TFDE}}, m, s, r$ )                                                                                      |
|------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 1: $\text{CRS}_{\text{FDE}} \leftarrow \text{FDE.Setup}(1^\lambda)$                                                                      | 1: <b>parse</b> $s, r$                                                                                                                    |
| 2: $\text{crs}_{\text{EQ}} \leftarrow \text{EQ.Setup}(1^\lambda)$                                                                        | 2: $\quad \quad \quad \parallel$ Sender-side thresholdization                                                                             |
| 3: <b>for</b> $(i, j) \in ([n_S] \times [n_R]) \cup \binom{[n_S]}{2} \cup \binom{[n_R]}{2}$ <b>do</b>                                    | 3: $x_S \leftarrow (s \  m); (\text{pp}_S, s_1^S, \dots, s_{n_S}^S) \leftarrow \text{UT.Setup}(1^\lambda, 1^d, (\mathbb{S}_S, n_S), x_S)$ |
| 4: $r_{i,j}^G \leftarrow \mathbb{S}\{0, 1\}^\lambda; (\text{pk}_{i,j}, \text{sk}_{i,j}) \leftarrow \text{NCE.Gen}(1^\lambda, r_{i,j}^G)$ | 4: $\quad \quad \quad \parallel$ Receiver-side thresholdization                                                                           |
| 5: $\text{CRS}_{\text{TFDE}} \leftarrow (\text{CRS}_{\text{FDE}}, \text{crs}_{\text{EQ}}, \{\text{pk}_{i,j}, \text{sk}_{i,j}\}_{i,j})$   | 5: $x_R \leftarrow r; (\text{pp}_R, s_1^R, \dots, s_{n_R}^R) \leftarrow \text{UT.Setup}(1^\lambda, 1^d, (\mathbb{S}_R, n_R), x_R)$        |
| 6: <b>return</b> $\text{CRS}_{\text{TFDE}}$                                                                                              | 6: $\quad \quad \quad \parallel$ Distribute shares over NCE channels                                                                      |
|                                                                                                                                          | 7: <b>for</b> $i \in [n_S]$ <b>do</b> $\text{ct}_i^S \leftarrow \text{NCE.Enc}(\text{pk}_{0,i}, s_i^S, r_{0,i}^E)$                        |
|                                                                                                                                          | 8: <b>for</b> $j \in [n_R]$ <b>do</b> $\text{ct}_j^R \leftarrow \text{NCE.Enc}(\text{pk}_{0,j}, s_j^R, r_{0,j}^E)$                        |
|                                                                                                                                          | 9: <b>return</b> $(\text{pp}_S, \{s_i^S\}_{i \in [n_S]}, \text{pp}_R, \{s_j^R\}_{j \in [n_R]})$                                           |

Fig. 4: TFDE Setup and Share Generation. The setup produces a compound CRS from FDE, EQ, and a full mesh of NCE key pairs.  $\text{ShareGen}$  thresholdizes the sender secret  $x_S = (s \| m)$  and receiver secret  $x_R = r$  via UT, which distributes shares over NCE channels.

*Threshold Encryption Rounds.* The three interactive rounds of the underlying FDE are executed in threshold structure, each following a uniform *commit-exchange-verify-combine* pattern. In Round 1, each active sender  $i \in A_S$  ( $|A_S| \geq t_S$ ) encodes  $P_1$  as a circuit  $\mathbb{C}_1$  satisfying  $\mathbb{C}_1(x_S) = P_1(s, m)$ , computes a partial evaluation  $y_i^1 \leftarrow \text{UT.Eval}(\text{pp}_S, s_i^S, \mathbb{C}_1)$ , commits via  $(c_i^1, d_i^1) \leftarrow \text{EQ.Commit}(1^\lambda, \text{crs}_{\text{EQ}}, (s_i^S, y_i^1))$ , broadcasts  $c_i^1$ , and transmits  $y_i^1$  to every co-sender over intra-sender NCE channels. After all parties verify received partials via  $\text{UT.Verify}$ , any party combines to obtain  $\mu_1 \leftarrow \text{UT.Combine}(\text{pp}_S, \{y_i^1\}_{i \in A_S})$ .



Round 2 mirrors this on the receiver side: each active receiver  $j \in A_R$  ( $|A_R| \geq t_R$ ) encodes  $P_2$  with  $\mu_1$  hardcoded as circuit  $\mathbb{C}_2$ , threshold-evaluates, commits, exchanges over intra-receiver NCE channels, verifies, and combines to produce  $\mu_2 \leftarrow \text{UT.Combine}(\text{pp}_R, \{y_j^2\}_{j \in A_R})$ . A designated combiner  $j^* \in A_R$  then transmits  $\mu_2$  to each sender party over cross-side NCE channels.

Round 3 repeats the sender-side pattern with circuit  $\mathbb{C}_3$  encoding  $P_3(s, m, \mu_1, \mu_2)$  (with  $\mu_1, \mu_2$  hardcoded), yielding  $\mu_3 \leftarrow \text{UT.Combine}(\text{pp}_S, \{y_i^3\}_{i \in A_S})$ , which a designated combiner  $i^* \in A_S$  sends to the receiver coalition over cross-side NCE channels. At every step, each party's internal state is locked behind an equivocal commitment and every transmission is wrapped in NCE, which creates a double layer of equivocability.

| TFDE.ThreshP <sub>1</sub> (CRS <sub>TFDE</sub> , pp <sub>S</sub> , {s <sub>i</sub> <sup>S</sup> } <sub>i ∈ A<sub>S</sub></sub> , m)                      | TFDE.ThreshP <sub>2</sub> (CRS <sub>TFDE</sub> , pp <sub>R</sub> , {s <sub>j</sub> <sup>R</sup> } <sub>j ∈ A<sub>R</sub></sub> , μ <sub>1</sub> )        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1: $\parallel A_S \subseteq [n_S],  A_S  \geq t_S$                                                                                                       | 1: $\parallel A_R \subseteq [n_R],  A_R  \geq t_R$                                                                                                       |
| 2: Define $\mathbb{C}_1(\cdot)$ s.t. $\mathbb{C}_1(x_S) = P_1(s, m)$ where $x_S = (s \  m)$                                                              | 2: Define $\mathbb{C}_2(\cdot)$ s.t. $\mathbb{C}_2(x_R) = P_2(r, \mu_1)$ with $\mu_1$ hardcoded                                                          |
| 3: <b>for</b> $i \in A_S$ <b>do</b>                                                                                                                      | 3: <b>for</b> $j \in A_R$ <b>do</b>                                                                                                                      |
| 4: $y_i^1 \leftarrow \text{UT.Eval}(\text{pp}_S, s_i^S, \mathbb{C}_1)$                                                                                   | 4: $y_j^2 \leftarrow \text{UT.Eval}(\text{pp}_R, s_j^R, \mathbb{C}_2)$                                                                                   |
| 5: $(c_i^1, d_i^1) \leftarrow \text{EQ.Commit}(1^\lambda, \text{crs}_{\text{EQ}}, (s_i^S, y_i^1))$                                                       | 5: $(c_j^2, d_j^2) \leftarrow \text{EQ.Commit}(1^\lambda, \text{crs}_{\text{EQ}}, (s_j^R, y_j^2))$ ; broadcast $c_j^2$                                   |
| 6: broadcast $c_i^1$                                                                                                                                     | 6: <b>for</b> $j' \in A_R \setminus \{j\}$ <b>do</b> NCE.Enc(pk <sub>j,j'</sub> , y <sub>j</sub> <sup>2</sup> , r <sub>j,j'</sub> <sup>E</sup> , 2) → j' |
| 7: <b>for</b> $i' \in A_S \setminus \{i\}$ <b>do</b> NCE.Enc(pk <sub>i,i'</sub> , y <sub>i</sub> <sup>1</sup> , r <sub>i,i'</sub> <sup>E</sup> , 1) → i' | 7: <b>for</b> $j \in A_R$ <b>do</b>                                                                                                                      |
| 8: <b>for</b> $i \in A_S$ <b>do</b>                                                                                                                      | 8: <b>for</b> $j' \in A_R \setminus \{j\}$ <b>do</b>                                                                                                     |
| 9: <b>for</b> $i' \in A_S \setminus \{i\}$ <b>do</b>                                                                                                     | 9: <b>assert</b> UT.Verify(pp <sub>R</sub> , y <sub>j'</sub> <sup>2</sup> , C <sub>2</sub> ) = 1                                                         |
| 10: <b>assert</b> UT.Verify(pp <sub>S</sub> , y <sub>i'</sub> <sup>1</sup> , C <sub>1</sub> ) = 1                                                        | 10: $\mu_2 \leftarrow \text{UT.Combine}(\text{pp}_R, \{y_j^2\}_{j \in A_R})$                                                                             |
| 11: $\mu_1 \leftarrow \text{UT.Combine}(\text{pp}_S, \{y_i^1\}_{i \in A_S})$                                                                             | 11: $\parallel$ Transmit $\mu_2$ to sender coalition via NCE                                                                                             |
| 12: <b>return</b> $\mu_1$                                                                                                                                | 12: <b>for</b> $i \in A_S$ <b>do</b> NCE.Enc(pk <sub>j*,i</sub> , μ <sub>2</sub> , r <sub>j*,i</sub> <sup>E</sup> , 2) → i; <b>return</b> $\mu_2$        |

Fig. 5: TFDE Threshold Rounds 1 and 2. Each party evaluates the FDE round function via UT.Eval, commits with EQ.Commit, exchanges partial evaluations over intra-side NCE channels, verifies via UT.Verify, and combines via UT.Combine. After Round 2, the combiner  $j^* \in A_R$  transmits  $\mu_2$  to the sender coalition over cross-side NCE channels.

*Threshold Decryption.* Upon receiving the complete transcript  $(\mu_1, \mu_2, \mu_3)$ , the receiver coalition encodes FDE.Dec with the full transcript hardcoded as circuit  $\mathbb{C}_{\text{dec}}$  and follows the same commit-exchange-verify-combine pattern: each receiver  $j \in A_R$  computes  $y_j^{\text{dec}} \leftarrow \text{UT.Eval}(\text{pp}_R, s_j^R, \mathbb{C}_{\text{dec}})$ , commits, exchanges, verifies, and combines to recover  $m/\perp \leftarrow \text{UT.Combine}(\text{pp}_R, \{y_j^{\text{dec}}\}_{j \in A_R})$ .

*Deniability under Coercion.* When a coercer targets a subset of senders  $C_S \subseteq A_S$  ( $|C_S| < t_S$ ) or receivers  $C_R \subseteq A_R$  ( $|C_R| < t_R$ ), the faking procedures activate three coordinated layers of equivocation. *First*, at the FDE level, FDE.SFake (resp. FDE.RFake) produces fake master randomness  $s'$  (resp.  $r'$ ) consistent with a cover message  $\hat{m}$ . *Second*, at the threshold level, the fake master secret  $\hat{x}_S = (s' \| \hat{m})$  (resp.  $\hat{x}_R = r'$ ) is used to re-simulate each coerced party's share via ShareSim; since  $|C_\sigma| < t_\sigma$ , the secret sharing's information-theoretic privacy guarantees [31] that the restricted shares are identically distributed regardless of the underlying secret. Each coerced party then recomputes its partial evaluations under the fake shares. *Third*, at the communication level, each coerced party's commitments are equivocated via EQ.Equiv<sub>1</sub>/EQ.Equiv<sub>2</sub>.

to open to the fake shares and fake partial evaluations, and each NCE ciphertext involving a coerced party is equivocated via  $\text{NCE.Sim}_2$  to be consistent with the fake data.  $\text{ThreshRFake}$  requires neither the real receiver randomness  $r$ , the real plaintext  $m$ , nor the real receiver shares, which preserves public receiver deniability: any entity can produce fake receiver views given only the public transcript and the fake plaintext.

| TFDE.ThreshP <sub>3</sub> (CRS <sub>TFDE</sub> , pp <sub>S</sub> , {s <sub>i</sub> <sup>S</sup> } <sub>i ∈ A<sub>S</sub></sub> , m, μ <sub>1</sub> , μ <sub>2</sub> )                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | TFDE.ThreshDec(CRS <sub>TFDE</sub> , pp <sub>R</sub> , {s <sub>j</sub> <sup>R</sup> } <sub>j ∈ A<sub>R</sub></sub> , μ <sub>1</sub> , μ <sub>2</sub> , μ <sub>3</sub> )                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1: Define $\mathbb{C}_3(\cdot)$ s.t. $\mathbb{C}_3(x_S) = P_3(s, m, \mu_1, \mu_2)$ with $\mu_1, \mu_2$ hardcoded<br>2: <b>for</b> $i \in A_S$ <b>do</b><br>3: $y_i^3 \leftarrow \text{UT.Eval}(\text{pp}_S, s_i^S, \mathbb{C}_3)$<br>4: $(c_i^3, d_i^3) \leftarrow \text{EQ.Commit}(1^\lambda, \text{crs}_{\text{EQ}}, (s_i^S, y_i^3))$<br>5:   broadcast $c_i^3$<br>6: <b>for</b> $i' \in A_S \setminus \{i\}$ <b>do</b> $\text{NCE.Enc}(\text{pk}_{i,i'}, y_i^3, r_{i,i',3}^E) \rightarrow i'$<br>7: <b>for</b> $i' \in A_S$ <b>do</b><br>8: <b>for</b> $i'' \in A_S \setminus \{i\}$ <b>do</b><br>9: <b>assert</b> $\text{UT.Verify}(\text{pp}_S, y_{i''}^3, \mathbb{C}_3) = 1$<br>10: $\mu_3 \leftarrow \text{UT.Combine}(\text{pp}_S, \{y_i^3\}_{i \in A_S})$<br>11:   // Transmit $\mu_3$ to receiver coalition via NCE<br>12: <b>for</b> $j \in A_R$ <b>do</b> $\text{NCE.Enc}(\text{pk}_{i^*,j}, \mu_3, r_{i^*,j,3}^E) \rightarrow j$<br>13: <b>return</b> $\mu_3$ | 1: Define $\mathbb{C}_{\text{dec}}(\cdot)$ s.t. $\mathbb{C}_{\text{dec}}(x_R) = \text{FDE.Dec}(r, \mu_1, \mu_2, \mu_3)$<br>2:   with $(\mu_1, \mu_2, \mu_3)$ hardcoded<br>3: <b>for</b> $j \in A_R$ <b>do</b><br>4: $y_j^{\text{dec}} \leftarrow \text{UT.Eval}(\text{pp}_R, s_j^R, \mathbb{C}_{\text{dec}})$<br>5: $(c_j^{\text{dec}}, d_j^{\text{dec}}) \leftarrow \text{EQ.Commit}(1^\lambda, \text{crs}_{\text{EQ}}, (s_j^R, y_j^{\text{dec}}))$<br>6:   broadcast $c_j^{\text{dec}}$<br>7: <b>for</b> $j' \in A_R \setminus \{j\}$ <b>do</b> $\text{NCE.Enc}(\text{pk}_{j,j'}, y_j^{\text{dec}}, r_{j,j',\text{dec}}^E) \rightarrow j'$<br>8: <b>for</b> $j \in A_R$ <b>do</b><br>9: <b>for</b> $j' \in A_R \setminus \{j\}$ <b>do</b><br>10: <b>assert</b> $\text{UT.Verify}(\text{pp}_R, y_{j'}^{\text{dec}}, \mathbb{C}_{\text{dec}}) = 1$<br>11: $m/\perp \leftarrow \text{UT.Combine}(\text{pp}_R, \{y_j^{\text{dec}}\}_{j \in A_R})$<br>12: <b>return</b> $m/\perp$ |

Fig. 6: TFDE Round 3 and Decryption. Round 3 mirrors Round 1 on the sender side with circuit  $\mathbb{C}_3$  encoding  $P_3$ ; the combiner  $i^* \in A_S$  sends  $\mu_3$  to receivers. Decryption encodes  $\text{FDE.Dec}$  as  $\mathbb{C}_{\text{dec}}$  and follows the same commit-exchange-verify-combine procedure.

## 4 Security Analysis of TFDE

As a threshold cryptosystem that is fully deniable, the TFDE must satisfy properties of both TPKE (Section A.3) and FDE (Section 2).

But there is one gap remaining here: There is a need for TFDE to capture deniability at the threshold granularity, where the adversary sees views of individual parties, not monolithic randomness. To address this, we provide properties of TFDE in the intersection of TPKE and FDE in the following to capture what it means for the composition to be deniable.

A coercer might want to target subsets of parties, see partial internal states (shares, partial evaluations, commitment openings, NCE randomness), and try to distinguish real from fake at the per-party level. The FDE guarantee that "*fake  $s$  is indistinguishable from real  $s$* " does not automatically imply "*fake share of  $s$  held by party  $i$ , together with party  $i$ 's commitment openings and NCE channel randomness, is indistinguishable from the real version.*" So, we lift the properties of FDE to threshold analogues in the following.

We bring the new properties that TFDE should have as a threshold multi-party protocol. We assume a Threshold Fully Deniable Encryption protocol as the  $\text{TFDE} = (\text{Setup}, \text{ShareGen}, \text{ThreshP}_1, \text{ThreshP}_2, \text{ThreshP}_3, \text{ThreshDec}, \text{ThreshSFake}, \text{ThreshRFake})$  with sender access structure  $(\mathbb{S}_S, n_S)$ , threshold  $t_S$ , receiver access structure  $(\mathbb{S}_R, n_R)$ , threshold  $t_R$ , and message space  $\mathcal{M}$ . Let  $A_S \subseteq [n_S]$ ,  $A_R \subseteq [n_R]$  be the active sender and receiver sets. For a set of parties  $S$ , write  $\text{view}_S$  for the joint view of parties in

| TFDE.ThreshSFake( $\text{CRS}_{\text{TFDE}}, \text{pp}_S, \{s_i^S\}_{i \in C_S}, m, \hat{m}, \mu_1, \mu_2, \mu_3$ )            | TFDE.ThreshRFake( $\text{CRS}_{\text{TFDE}}, \text{pp}_R, C_R, \hat{m}, \mu_1, \mu_2, \mu_3; \rho$ )                           |
|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| 1: $\parallel C_S \subseteq A_S,  C_S  < t_S$                                                                                  | 1: $\parallel C_R \subseteq A_R,  C_R  < t_R$                                                                                  |
| 2: $\parallel$ FDE-level sender faking                                                                                         | 2: $\parallel$ FDE-level receiver faking                                                                                       |
| 3: $s' \leftarrow \text{FDE.SFake}(s, m, \hat{m}, \mu_1, \mu_2, \mu_3)$                                                        | 3: $r' \leftarrow \text{FDE.RFake}(\hat{m}, \mu_1, \mu_2, \mu_3; \rho)$                                                        |
| 4: $\parallel$ Fake share construction                                                                                         | 4: $\parallel$ Fake share construction                                                                                         |
| 5: $\hat{x}_S \leftarrow (s' \parallel \hat{m})$                                                                               | 5: $\hat{x}_R \leftarrow r'$                                                                                                   |
| 6: <b>for</b> $i \in C_S$ <b>do</b> $s_i^{S'} \leftarrow \text{ShareSim}_S(C_S, \hat{x}_S)$                                    | 6: <b>for</b> $j \in C_R$ <b>do</b> $s_j^{R'} \leftarrow \text{ShareSim}_R(C_R, \hat{x}_R)$                                    |
| 7: $\parallel$ Fake partial evaluations                                                                                        | 7: $\parallel$ Fake partial evaluations                                                                                        |
| 8: <b>for</b> $i \in C_S$ <b>do</b>                                                                                            | 8: <b>for</b> $j \in C_R$ <b>do</b>                                                                                            |
| 9: <b>for</b> $\ell \in \{1, 3\}$ <b>do</b>                                                                                    | 9: <b>for</b> $\ell \in \{2, \text{dec}\}$ <b>do</b>                                                                           |
| 10: $\hat{y}_i^\ell \leftarrow \text{UT.Eval}(\text{pp}_S, s_i^{S'}, \mathbb{C}_\ell)$                                         | 10: $\hat{y}_j^\ell \leftarrow \text{UT.Eval}(\text{pp}_R, s_j^{R'}, \mathbb{C}_\ell)$                                         |
| 11: $\parallel$ Equivocate commitments                                                                                         | 11: $\parallel$ Equivocate commitments                                                                                         |
| 12: <b>for</b> $i \in C_S$ <b>do</b>                                                                                           | 12: <b>for</b> $j \in C_R$ <b>do</b>                                                                                           |
| 13: <b>for</b> $\ell \in \{1, 3\}$ <b>do</b>                                                                                   | 13: <b>for</b> $\ell \in \{2, \text{dec}\}$ <b>do</b>                                                                          |
| 14: $(\widehat{\text{crs}}_i, \hat{c}_i^\ell, \text{st}_i^\ell) \leftarrow \text{EQ.Equiv}_1(1^\lambda)$                       | 14: $(\widehat{\text{crs}}_j, \hat{c}_j^\ell, \text{st}_j^\ell) \leftarrow \text{EQ.Equiv}_1(1^\lambda)$                       |
| 15: $d_i^{\ell'} \leftarrow \text{EQ.Equiv}_2(1^\lambda, \text{st}_i^\ell, (s_i^{S'}, \hat{y}_i^\ell))$                        | 15: $d_j^{\ell'} \leftarrow \text{EQ.Equiv}_2(1^\lambda, \text{st}_j^\ell, (s_j^{R'}, \hat{y}_j^\ell))$                        |
| 16: $\parallel$ Equivocate NCE channels                                                                                        | 16: $\parallel$ Equivocate NCE channels                                                                                        |
| 17: <b>for</b> $i \in C_S$ <b>do</b>                                                                                           | 17: <b>for</b> $j \in C_R$ <b>do</b>                                                                                           |
| 18: <b>for each</b> NCE channel involving $i$ <b>do</b>                                                                        | 18: <b>for each</b> NCE channel involving $j$ <b>do</b>                                                                        |
| 19: $(\hat{r}_{i,\cdot}^G, \hat{r}_{i,\cdot}^E) \leftarrow \text{NCE.Sim}_2(\hat{y}_i^\ell, \text{st}_{i,\cdot}^{\text{NCE}})$ | 19: $(\hat{r}_{j,\cdot}^G, \hat{r}_{j,\cdot}^E) \leftarrow \text{NCE.Sim}_2(\hat{y}_j^\ell, \text{st}_{j,\cdot}^{\text{NCE}})$ |
| 20: <b>return</b> $\{(s_i^{S'}, d_i^{\ell'}, d_i^{3'}, \hat{r}_{i,\cdot}^G, \hat{r}_{i,\cdot}^E)\}_{i \in C_S}$                | 20: <b>return</b> $\{(s_j^{R'}, d_j^{\ell'}, d_j^{\text{dec}}, \hat{r}_{j,\cdot}^G, \hat{r}_{j,\cdot}^E)\}_{j \in C_R}$        |

Fig. 7: Threshold Sender Faking and Threshold Receiver Faking. **ThreshRFake** takes neither the real receiver randomness  $r$ , the real plaintext  $m$ , nor the real receiver shares  $\{s_j^R\}$ , which preserves public receiver deniability.

$S$ , consisting of their UT shares, partial evaluations, equivocable commitment openings, NCE ciphertexts and randomness, and all protocol messages received. Write  $\text{view}_S$  for the fake view produced by **ThreshSFake** or **ThreshRFake**.

**Definition 6 (Threshold Bideniability).** TFDE is threshold bideniable if no PPT adversary  $\mathcal{A}$  has more than negligible advantage in the following game, for any  $m_0, m_1 \in \mathcal{M}$ :

1. The challenger runs  $\text{CRS}_{\text{TFDE}} \leftarrow \text{Setup}(1^\lambda)$ , generates shares via **ShareGen**, and chooses  $b \leftarrow \{0, 1\}$ .
2. The challenger executes the three threshold rounds with plaintext  $m_b$  to obtain transcript  $\text{tr}^* = (\mu_1, \mu_2, \mu_3)$  and all parties' internal states.
3. The challenger sends  $(\text{CRS}_{\text{TFDE}}, \text{pp}_S, \text{pp}_R, \text{tr}^*)$  to  $\mathcal{A}$ .  $\mathcal{A}$  outputs coercion sets  $(C_S, C_R)$  with  $|C_S| < t_S$ ,  $|C_R| < t_R$ .
4.
  - If  $b = 0$ : The challenger gives  $\mathcal{A}$  the true views  $(\text{view}_{C_S}, \text{view}_{C_R})$  from the execution with plaintext  $m_0$ .
  - If  $b = 1$ : The challenger computes fake views
 
$$\{\widetilde{\text{view}}_i\}_{i \in C_S} \leftarrow \text{ThreshSFake}(\text{CRS}_{\text{TFDE}}, \text{msk}, \text{pp}_S, \{s_i^S\}_{i \in C_S}, m_1, m_0, \text{tr}^*),$$

$$\{\widetilde{\text{view}}_j\}_{j \in C_R} \leftarrow \text{ThreshRFake}(\text{CRS}_{\text{TFDE}}, \text{msk}, \text{pp}_R, C_R, m_0, \text{tr}^*; \rho),$$
 and gives  $\mathcal{A}$  the fake views  $(\{\widetilde{\text{view}}_i\}_{i \in C_S}, \{\widetilde{\text{view}}_j\}_{j \in C_R})$ .
5.  $\mathcal{A}$  outputs  $b'$  and wins if  $b' = b$ .

For all PPT  $\mathcal{A}$ , all  $m_0, m_1 \in \mathcal{M}$ :  $|\Pr[b' = b] - \frac{1}{2}| \leq \text{negl}(\lambda)$ .

The Definition 6 subsumes:

- $C_S, C_R$  are chosen *after* seeing  $\text{tr}^*$  and public parameters (adaptive party selection).
- setting  $C_R = \emptyset$  (resp.  $C_S = \emptyset$ ) recovers the per-side statement (intra-side fake view indistinguishability).
- the challenger produces  $\widetilde{\text{view}}_{C_S}$  and  $\widetilde{\text{view}}_{C_R}$  jointly using the shared  $\text{msk}$ ; mutual consistency is a correctness requirement on the faking procedures (verified by the challenger), not a separate security property (cross-side fake consistency).
- the level switch from  $\ell=0$  to  $\ell=1$  is an internal hybrid step in the proof of this definition, not an externally observable property (threshold level indistinguishability).

**Definition 7 (Threshold Off-the-Record Deniability).** TFDE is threshold off-the-record deniable if no PPT adversary  $\mathcal{A}$  has more than negligible advantage in the following game, for any  $m_0, m_1, m_2 \in \mathcal{M}$ :

1. The challenger runs  $\text{CRS}_{\text{TFDE}} \leftarrow \text{Setup}(1^\lambda)$ , generates shares, and chooses  $b \leftarrow \{0, 1, 2\}$ .
2. Let  $m^*$  be the real plaintext determined by  $b$ :  $m^* = \begin{cases} m_0 & \text{if } b=0, \\ m_1 & \text{if } b=1, \\ m_2 & \text{if } b=2. \end{cases}$  The challenger executes the protocol with plaintext  $m^*$  to obtain  $\text{tr}^*$ .
3. The challenger sends  $(\text{CRS}_{\text{TFDE}}, \text{pp}_S, \text{pp}_R, \text{tr}^*)$  to  $\mathcal{A}$ .  $\mathcal{A}$  outputs  $(C_S, C_R)$  with  $|C_S| < t_S$ ,  $|C_R| < t_R$ .
4.
  - $b=0$  (sender truthful, receiver deviates): Give  $\mathcal{A}$  true sender views  $\text{view}_{C_S}$  and fake receiver views  $\{\widetilde{\text{view}}_j\}_{j \in C_R} \leftarrow \text{ThreshRFake}(\dots, m_1, \text{tr}^*; \rho)$ .
  - $b=1$  (sender deviates, receiver truthful): Give  $\mathcal{A}$  fake sender views  $\{\widetilde{\text{view}}_i\}_{i \in C_S} \leftarrow \text{ThreshSFake}(\dots, m_1, m_0, \text{tr}^*)$  and true receiver views  $\text{view}_{C_R}$ .
  - $b=2$  (both deviate, possibly inconsistently): Give  $\mathcal{A}$  fake sender views as the values  $\{\widetilde{\text{view}}_i\}_{i \in C_S} \leftarrow \text{ThreshSFake}(\dots, m_2, m_0, \text{tr}^*)$  and fake receiver views  $\{\widetilde{\text{view}}_j\}_{j \in C_R} \leftarrow \text{ThreshRFake}(\dots, m_1, \text{tr}^*; \rho)$ .
5.  $\mathcal{A}$  outputs  $b'$  and wins if  $b' = b$ .

For all PPT  $\mathcal{A}$ , all  $m_0, m_1, m_2 \in \mathcal{M}$ :  $|\Pr[b' = b] - \frac{1}{3}| \leq \text{negl}(\lambda)$ .

As in Definition 6, adaptive party selection is built in, and cross-side consistency is enforced by the shared  $\text{msk}$  rather than stated as a separate property.

**Definition 8 (Threshold Public Receiver Deniability).** TFDE has threshold public receiver deniability if  $\text{ThreshRFake}$  has the following restricted syntax:

$$\{\widetilde{\text{view}}_j\}_{j \in C_R} \leftarrow \text{ThreshRFake}(\text{CRS}_{\text{TFDE}}, \text{msk}, \text{pp}_R, C_R, \hat{m}, \mu_1, \mu_2, \mu_3; \rho),$$

where  $\text{msk}$  is system-level secret state (equivocation trapdoor and NCE simulation states), not receiver-specific secret state. In particular,  $\text{ThreshRFake}$  does not take as input:

- the real receiver master randomness  $r$ ,
- the real plaintext  $m$ , or
- the real receiver shares  $\{s_j^R\}_{j \in C_R}$ .

This makes sure that any entity with access to  $\text{msk}$ , not just the coerced receivers, can produce fake receiver views without knowledge of any receiver-specific secret.

**Definition 9 (Full Threshold Deniability).** A TFDE protocol is fully threshold deniable if it satisfies:

1. Threshold bideniability (Definition 6),
2. Threshold off-the-record deniability (Definition 7), and
3. Threshold public receiver deniability (Definition 8),

in addition to the standard TPKE properties (Definitions 13, 14, 15, and 16).

Now by having necessary definitions for the security of TFDE, in the following theorems we state that proposed TFDE protocol is fully threshold deniable.

**Theorem 1.** The TFDE protocol (Definition 5) satisfies threshold bideniability (Definition 6) under the assumptions that FDE satisfies bideniability [16], UT supports share simulation relative to fixed public parameters, EQ satisfies CRS indistinguishability and equivocability, and NCE satisfies simulatability.

**Theorem 2.** Our TFDE (Definition 5) satisfies threshold off-the-record deniability (Definition 7) under the same assumptions as Theorem 1, with the additional requirement that FDE satisfies off-the-record deniability [16].

The three cases ( $b \in \{0, 1, 2\}$ ) are reduced to the underlying FDE off-the-record deniability game by first invoking UT share simulation, EQ equivocability, and NCE simulatability to replace each coerced party's threshold-layer state, then appealing to the FDE-level three-way indistinguishability to conclude.

**Theorem 3.** TFDE protocol (Definition 5) satisfies threshold public receiver deniability (Definition 8).

By inspection of ThreshRFake (Figure 7), it takes as input the values of the  $(\text{CRS}_{\text{TFDE}}, \text{msk}, \text{pp}_R, C_R, \hat{m}, \mu_1, \mu_2, \mu_3; \rho)$ : it invokes  $\text{FDE.RFake}(\hat{m}, \mu_1, \mu_2, \mu_3; \rho)$ , which by the public receiver deniability of FDE [16] requires neither  $r$  nor  $m$ ; and  $\text{ShareSim}_R(C_R, \hat{x}_R)$  requires only the target secret and the coerced set, not the real shares.

**Theorem 4.** The TFDE (Definition 5) is fully threshold deniable (Definition 9).

This follows directly from Theorems 5–8 (standard TPKE properties) and Theorems 1–3 (threshold deniability properties).

## 5 Strong Off-the-Record Deniability

We formalize Off-the-Record Deniability in Figure 8 to recall its concept; however, off-the-record deniability leaves unaddressed an important case: when both parties coordinate to claim a consistent fake plaintext that differs from the true message—for instance, when both parties are coerced simultaneously and choose to provide the same fabricated explanation to avoid suspicion or detection of their deception [16]. While standard bideniability [15] covers this scenario, it assumes both parties execute the protocol honestly and only lie afterward. Off-the-record deniability, conversely, considers arbitrary deviations but only guarantees protection when the parties' claims are inconsistent.

**Definition 10 (Off-the-Record Deniability).** An interactive deniable encryption scheme  $\Pi$  for message space  $\mathcal{M}$  satisfies the property of off-the-record deniability if for any efficient adversary  $\mathcal{A}$ , the advantage of the value  $\text{Adv}_{\mathcal{A}}^{\text{OTR}}(\lambda) = \max_{i \neq j \in \{0,1,2\}} |\Pr[\text{Expt}_{\mathcal{A},i}^{\text{OTR}}(1^\lambda) = 1] - \Pr[\text{Expt}_{\mathcal{A},j}^{\text{OTR}}(1^\lambda) = 1]|$  is negligible in the  $\lambda$ , where experiments  $\text{Expt}_{\mathcal{A},b}^{\text{OTR}}(1^\lambda)$  for  $b \in \{0,1,2\}$  are defined in Figure 8.

| $\text{Expt}_{\mathcal{A},0}^{\text{OTR}}(1^\lambda, m_0, m_1, m_2)$             | $\text{Expt}_{\mathcal{A},1}^{\text{OTR}}(1^\lambda, m_0, m_1, m_2)$             | $\text{Expt}_{\mathcal{A},2}^{\text{OTR}}(1^\lambda, m_0, m_1, m_2)$            |
|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| 1: $b \leftarrow 0$                                                              | 1: $b \leftarrow 1$                                                              | 1: $b \leftarrow 2$                                                             |
| 2: $r_{\text{setup}} \leftarrow \{0,1\}^{ r_{\text{setup}} }$                    | 2: $r_{\text{setup}} \leftarrow \{0,1\}^{ r_{\text{setup}} }$                    | 2: $r_{\text{setup}} \leftarrow \{0,1\}^{ r_{\text{setup}} }$                   |
| 3: $\text{CRS} \leftarrow \text{Setup}(1^\lambda; r_{\text{setup}})$             | 3: $\text{CRS} \leftarrow \text{Setup}(1^\lambda; r_{\text{setup}})$             | 3: $\text{CRS} \leftarrow \text{Setup}(1^\lambda; r_{\text{setup}})$            |
| 4: $s^* \leftarrow \{0,1\}^{ s }; r^* \leftarrow \{0,1\}^{ r }$                  | 4: $s^* \leftarrow \{0,1\}^{ s }; r^* \leftarrow \{0,1\}^{ r }$                  | 4: $s^* \leftarrow \{0,1\}^{ s }; r^* \leftarrow \{0,1\}^{ r }$                 |
| 5: $\mu_1^* \leftarrow P_1(s^*, m_0); \mu_2^* \leftarrow P_2(r^*, \mu_1^*)$      | 5: $\mu_1^* \leftarrow P_1(s^*, m_1); \mu_2^* \leftarrow P_2(r^*, \mu_1^*)$      | 5: $\mu_1^* \leftarrow P_1(s^*, m_2); \mu_2^* \leftarrow P_2(r^*, \mu_1^*)$     |
| 6: $\mu_3^* \leftarrow P_3(s^*, m_0, \mu_1^*, \mu_2^*)$                          | 6: $\mu_3^* \leftarrow P_3(s^*, m_1, \mu_1^*, \mu_2^*)$                          | 6: $\mu_3^* \leftarrow P_3(s^*, m_2, \mu_1^*, \mu_2^*)$                         |
| 7: $\text{tr}^* \leftarrow (\mu_1^*, \mu_2^*, \mu_3^*)$                          | 7: $\text{tr}^* \leftarrow (\mu_1^*, \mu_2^*, \mu_3^*)$                          | 7: $\text{tr}^* \leftarrow (\mu_1^*, \mu_2^*, \mu_3^*)$                         |
| 8: $\rho_R \leftarrow \{0,1\}^{ \rho_R }$                                        | 8: $\rho_S \leftarrow \{0,1\}^{ \rho_S }$                                        | 8: $\rho_S \leftarrow \{0,1\}^{ \rho_S }; \rho_R \leftarrow \{0,1\}^{ \rho_R }$ |
| 9: $r' \leftarrow \text{RFake}(r^*, m_0, m_1, \text{tr}^*; \rho_R)$              | 9: $s' \leftarrow \text{SFake}(s^*, m_1, m_0, \text{tr}^*; \rho_S)$              | 9: $s' \leftarrow \text{SFake}(s^*, m_2, m_0, \text{tr}^*; \rho_S)$             |
| 10: $b' \leftarrow \mathcal{A}(\text{CRS}, m_0, m_1, m_2, s^*, r', \text{tr}^*)$ | 10: $b' \leftarrow \mathcal{A}(\text{CRS}, m_0, m_1, m_2, s', r^*, \text{tr}^*)$ | 10: $r' \leftarrow \text{RFake}(r^*, m_2, m_1, \text{tr}^*; \rho_R)$            |
| 11: <b>return</b> $b'$                                                           | 11: <b>return</b> $b'$                                                           | 11: $b' \leftarrow \mathcal{A}(\text{CRS}, m_0, m_1, m_2, s', r', \text{tr}^*)$ |
|                                                                                  |                                                                                  | 12: <b>return</b> $b'$                                                          |

Fig. 8: Interactive Deniable Encryption's Off-the-Record Deniability Experiments

This gap becomes bigger in practical settings where parties may have opportunities to coordinate their responses under coercion, or where they face incentives to present a unified false narrative. In such cases, the adversary observes both parties consistently claiming the same fake plaintext, which should appear indistinguishable from a scenario where they consistently claim different fake plaintexts or where their claims conflict entirely. Without this guarantee, the mere consistency of the parties' claims could itself leak information about the true communication.

Therefore, to fill this gap, we introduce Strong Off-the-record Deniability (SORD), which strengthens Definition 10 by adding a fourth indistinguishability case where both parties deviate with consistent claims. SORD ensures that no adversary can distinguish whether the observed consistency in the parties' claims reflects truth, coordination, or independent fabrication. Combined with bideniability, SORD protects all possible coercion scenarios and response patterns.

**Definition 11 (Strong Off-the-Record Deniability).** Let  $\pi = (\text{Setup}, P_1, P_2, P_3, \text{Dec}, \text{SFake}, \text{RFake})$  be a three-message interactive encryption scheme with message space  $\mathcal{M}$ . We strengthen Definition 10 by requiring indistinguishability under adaptive coercion (where parties may be coerced at any round during execution) and adding a fourth case where both parties deviate with consistent claims. We say that  $\pi$  satisfies strong off-the-record deniability (SORD) if no PPT adversary  $\mathcal{A}$  wins with more than negligible advantage in the following game, for any  $m_0, m_1, m_2 \in \mathcal{M}$ .

1. The challenger generates  $\text{CRS} \leftarrow \text{Setup}(r_{\text{Setup}})$  for random  $r_{\text{Setup}}$ , and chooses  $b \leftarrow \{0,1,2,3\}$ .

2. The challenger initializes the protocol execution with random  $s^*, r^*$  and tracks the current round  $\rho \in \{1, 2, 3\}$ .
3.  $\mathcal{A}$  adaptively issues coercion queries  $(\text{party}, \rho)$  where  $\text{party} \in \{\text{sender}, \text{receiver}\}$  and  $\rho \in \{1, 2, 3, 4\}$  indicates the round at which coercion occurs (with  $\rho = 4$  denoting post-execution coercion). Let  $\tau_S$  and  $\tau_R$  denote the coercion times for the sender and receiver, respectively (adaptive coercion phase).
4. Based on  $b$  and the coercion pattern  $(\tau_S, \tau_R)$ , the challenger responds as follows:
  - Sender truthful, Receiver deviates ( $b=0$ )
    - Generate  $\text{tr}^* \leftarrow \pi(s^*, r^*, m_0)$
    - Upon sender coercion at round  $\tau_S$ : give real partial state  $\text{st}_S^{\tau_S}(s^*, m_0)$
    - Upon receiver coercion at round  $\tau_R$ : Provide fake partial state  $\text{st}_R^{\tau_R} \leftarrow \text{RFake}_{\tau_R}(r^*, m_0, m_1, \text{tr}_{<\tau_R}; \rho_R)$  where  $\text{tr}_{<\tau_R}$  denotes the partial transcript up to round  $\tau_R$
  - Sender deviates, Receiver truthful ( $b=1$ )
    - Generate  $\text{tr}^* \leftarrow \pi(s^*, r^*, m_1)$
    - Upon sender coercion at round  $\tau_S$ : Provide fake partial state  $\text{st}_S^{\tau_S} \leftarrow \text{SFake}_{\tau_S}(s^*, m_1, m_0, \text{tr}_{<\tau_S}; \rho_S)$
    - Upon receiver coercion at round  $\tau_R$ : give real partial state  $\text{st}_R^{\tau_R}(r^*, m_1)$
  - Both parties deviate with inconsistent claims ( $b=2$ )
    - Generate  $\text{tr}^* \leftarrow \pi(s^*, r^*, m_2)$
    - Upon sender coercion at round  $\tau_S$ : Provide fake partial state  $\text{st}_S^{\tau_S} \leftarrow \text{SFake}_{\tau_S}(s^*, m_2, m_0, \text{tr}_{<\tau_S}; \rho_S)$
    - Upon receiver coercion at round  $\tau_R$ : Provide fake partial state  $\text{st}_R^{\tau_R} \leftarrow \text{RFake}_{\tau_R}(r^*, m_2, m_1, \text{tr}_{<\tau_R}; \rho_R)$
  - Both parties deviate with consistent fake claims ( $b=3$ )
    - Generate  $\text{tr}^* \leftarrow \pi(s^*, r^*, m_2)$
    - Upon sender coercion at round  $\tau_S$ : Provide fake partial state  $\text{st}_S^{\tau_S} \leftarrow \text{SFake}_{\tau_S}(s^*, m_2, m_0, \text{tr}_{<\tau_S}; \rho_S)$
    - Upon receiver coercion at round  $\tau_R$ : Provide fake partial state  $\text{st}_R^{\tau_R} \leftarrow \text{RFake}_{\tau_R}(r^*, m_2, m_0, \text{tr}_{<\tau_R}; \rho_R)$
5. After coercion, the protocol continues (if not yet complete) using the provided states. If a party is coerced mid-execution, the challenger continues the protocol execution using the fake state for that party while maintaining consistency with already-sent messages.
6.  $\mathcal{A}$  outputs  $b' \in \{0, 1, 2, 3\}$  and wins if  $b' = b$ .

Here,  $\text{st}_P^\rho$  denotes the internal state of party  $P \in \{\text{sender}, \text{receiver}\}$  at round  $\rho$ , which includes all random coins used up to that point, all messages sent and received, and any intermediate computation results. The faking algorithms  $\text{SFake}_\rho$  and  $\text{RFake}_\rho$  are round-specific variants that can produce fake states at any round  $\rho \in \{1, 2, 3, 4\}$  that are consistent with the partial transcript observed so far. The advantage of  $\mathcal{A}$  is defined as  $|\Pr[\mathcal{A} \text{ wins}] - 1/4|$ .

*Future Directions.* We consider designing a deniable protocol that can be strongly off-the-record deniable as a future work; a protocol that satisfies bideniability and strong off-the-record deniability can get referred as Strongly Fully Deniable Encryption. Also, the distributed key generation in sender-deniable protocol can get eliminated via silent-setup threshold cryptosystems [39]; however, current constructions rely on witness encryption [40], which remains inefficient [41, 42], and designing more efficient *silent-setup* threshold public key scheme is an open problem.



## A Appendix

### A.1 Computational Assumptions

*One-Way Functions* [43]. A function  $f : \{0,1\}^* \rightarrow \{0,1\}^*$  is a *one-way function* if it is polynomial-time computable, yet for every probabilistic polynomial-time adversary  $\mathcal{A}$  and all sufficiently large  $n \in \mathbb{N}$ ,  $\Pr[\mathcal{A}(1^n, f(x)) \in f^{-1}(f(x))] \leq \text{negl}(\lambda)(n)$ , where the probability is taken over  $x \leftarrow \{0,1\}^n$  and the internal randomness of  $\mathcal{A}$ .

*Subexponentially One-Way Functions* [44]. A function  $f : \{0,1\}^* \rightarrow \{0,1\}^*$  is a *subexponentially one-way function* if it is polynomial-time computable, yet for every adversary  $\mathcal{A}$  running in time  $2^{n^\epsilon}$  and some fixed  $\epsilon > 0$ , and all sufficiently large  $n \in \mathbb{N}$ ,  $\Pr[\mathcal{A}(1^n, f(x)) \in f^{-1}(f(x))] \leq 2^{-n^\epsilon}$ , where the probability is taken over  $x \leftarrow \{0,1\}^n$  and the internal randomness of  $\mathcal{A}$ .

*Learning With Errors* [45]. Let  $n, m, q$  be positive integers with  $q$  prime, and let  $\chi$  be an error distribution over  $\mathbb{Z}_q$ . The  $\text{LWE}_{n,m,q,\chi}$  decision problem requires distinguishing, with non-negligible advantage, the distribution  $\{(\mathbf{a}_i, \mathbf{a}_i^\top \mathbf{s} + e_i)\}$  from  $\{(\mathbf{a}_i, u_i)\}$ , where  $\mathbf{a}_i \leftarrow \mathbb{Z}_q^n$ ,  $\mathbf{s} \leftarrow \mathbb{Z}_q^m$ ,  $e_i \leftarrow \chi$ , and  $u_i \leftarrow \mathbb{Z}_q$ .

*Ring Short Integer Solution* [46]. Let  $R_q = \mathbb{Z}_q[X]/(f(X))$  for an irreducible polynomial  $f \in \mathbb{Z}[X]$  of degree  $n$ . The  $\text{RSIS}_{n,q,\beta}$  problem requires finding a non-zero element  $z \in R_q$  satisfying  $a \cdot z = 0$  in  $R_q$  and  $\|z\| \leq \beta$ , given  $a \leftarrow R_q$ , where  $\|\cdot\|$  denotes the Euclidean norm of the coefficient vector.

### A.2 Instantiations and Parameters

*Universal Thresholdizer*. We adopt the UT from Boneh et al. [31], built upon Linear Secret Sharing Scheme  $\{0,1\}$ -LSSS [31], Fully Homomorphic Encryption FHE [47] (together with  $\{0,1\}$ -LSSS they build TFHE), Pre-processing Zero-knowledge Proof PZK [48], Commitment scheme Com [49], which is merged by lattice-based UT [29] and  $\forall(\mathbb{S}, N) : \text{TFHE}_{\mathbb{A}} | \mathbb{A} \subseteq 2^{\mathbb{S}}$  from Jain et al. [30]. We have formalized in Figure 9 how this UT operates, instantiated with  $n_S, n_R \leq N$ , depth bound  $d$  sufficient for the FDE circuits  $\mathbb{C}_1, \mathbb{C}_2, \mathbb{C}_3, \mathbb{C}_{\text{dec}}$ , and share sizes  $|s_i| \leq \text{poly}(\lambda, d, N)$  which achieves  $\lambda$ -bit security under  $\text{LWE}_{n,m,q,\chi}$  [45] assumption.

*Non-Committing Encryption*. We instantiate NCE from Brakerski, Branco, Döttling, Garg, and Malavolta [50], which achieves constant ciphertext-rate  $O(1)$  in the plain model (no trusted setup) from the hardness of  $\text{LWE}_{k,n,q,\chi}$  with superpolynomial modulus-to-noise ratio  $q/B = \lambda^{\omega(1)}$  [51], instantiated with message length  $\ell$ , packed Regev public key  $(A \in \mathbb{Z}_q^{k \times n}, \{v_i\}_{i \in [\ell]})$ , compressed ciphertext  $(c_1 \in \mathbb{Z}_q^n, \{c_{2,i}\}_{i \in [\ell]} \in \{0,1\}^\ell, z \in \mathbb{Z}_q)$ , error-correcting code of constant rate  $R = O(1)$  and error rate  $1/2 - \delta$  for fixed  $\delta > 0$  [52], subset sizes  $|I_R| = |I_S| = \ell/8$ , and share size  $|I_g| = 3\ell/4$ , achieving  $\lambda$ -bit security and overall ciphertext-rate  $O(1)$  as  $\ell \rightarrow \infty$ .

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>UT.Setup</b> ( $1^\lambda, 1^d, \mathbb{A}, x$ )                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | <b>UT.Eval</b> ( $pp, s_i, \mathbb{C}$ )                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 1: $(\text{tfhepk}, \text{tfhesk}_1, \dots, \text{tfhesk}_N) \leftarrow \text{TFHE.Setup}(1^\lambda, 1^d, \mathbb{A})$<br>2: $\forall i \in [k]. \text{ct}_i \leftarrow \text{TFHE.Encrypt}(\text{tfhepk}, x_i)$<br>3: $\forall i \in [N]. (\sigma_{V,i}, \sigma_{P,i}) \leftarrow \text{PZK.Pre}(1^\lambda)$<br>4: $\forall i \in [N]. r_i \xleftarrow{\mathbb{R}} \{0,1\}^\lambda$<br>5: $\forall i \in [N]. \text{com}_i \leftarrow \text{Com}(\text{tfhesk}_i; r_i)$<br>6: $pp \leftarrow (\text{tfhepk}, \{\text{ct}_i\}_{i \in [k]}, \{\sigma_{V,i}\}_{i \in [N]}, \{\text{com}_i\}_{i \in [N]})$<br>7: $\forall i \in [N]. s_i \leftarrow (\text{tfhesk}_i, \sigma_{P,i}, r_i)$<br>8: <b>return</b> $(pp, s_1, \dots, s_N)$ | 1: $\hat{\text{ct}} \leftarrow \text{TFHE.Eval}(\text{tfhepk}, \mathbb{C}, \{\text{ct}_i\}_{i=1}^k)$<br>2: $p_i \leftarrow \text{TFHE.PartDec}(\text{tfhepk}, \hat{\text{ct}}, \text{tfhesk}_i)$<br>3: $\Psi_i : \exists(\text{tfhesk}_i, r_i) \mid \text{com}_i \leftarrow \text{Com}(\text{tfhesk}_i; r_i)$<br>$\wedge p_i \leftarrow \text{TFHE.PartDec}(\text{tfhepk}, \hat{\text{ct}}, \text{tfhesk}_i)$<br>4: $\pi_i \leftarrow \text{PZK.Prove}(\sigma_{P,i}, \Psi_i, (\text{tfhesk}_i, r_i))$<br>5: $y_i \leftarrow (p_i, \pi_i)$<br>6: <b>return</b> $y_i$ |
| <b>UT.Combine</b> ( $pp, B$ )                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | <b>UT.Verify</b> ( $pp, y_i, \mathbb{C}$ )                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 1: $B \leftarrow \{y_i\}_{i \in S} \mid (S \subseteq \{1, \dots, N\}) \vee (S \in \mathbb{A})$<br>2: $\forall i \in S: y_i \leftarrow (p_i, \pi_i)$<br>3: $y \leftarrow \text{TFHE.FinDec}(\text{tfhepk}, \{p_i\}_{i \in S})$<br>4: <b>return</b> $y$                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | 1: $\hat{\text{ct}} \leftarrow \text{TFHE.Eval}(\text{tfhepk}, \mathbb{C}, \{\text{ct}_i\}_{i=1}^k)$<br>2: $\Psi_i : \exists(\text{tfhesk}_i, r_i) \mid \text{com}_i \leftarrow \text{Com}(\text{tfhesk}_i; r_i)$<br>$\wedge p_i \leftarrow \text{TFHE.PartDec}(\text{tfhepk}, \hat{\text{ct}}, \text{tfhesk}_i)$<br>3: $y_i \leftarrow (p_i, \pi_i)$<br>4: <b>return</b> $\text{PZK.Verify}(\sigma_{V,i}, \Psi_i, \pi_i)$                                                                                                                                          |

Fig. 9: Universal Thresholdizer from TFHE = (Setup, Encrypt, Eval, PartDec, FinDec) [47], PZK = (PZK.Pre, PZK.Prove, PZK.Verify) [48], and Com = (Com) [49]

*Fully Deniable Encryption.* We adopt the FDE of Canetti et al. [16], which operates in the CRS model and achieves full deniability for a single-bit message space  $\mathcal{M} = \{0,1\}$  under subexponentially secure indistinguishability obfuscation (iO) and subexponentially secure one-way functions [14], with a CRS consisting of six obfuscated programs and a three-message interaction, where the receiver’s deniability is public (i.e., RFake does not require the true receiver randomness  $r^*$ ) and a single CRS suffices for an unbounded number of encryptions.

*Equivocable Commitment.* For EQ, we instantiate the locally-equivocable commitment scheme with all-but-one binding of Rotem and Segev [32] in a 256-bit prime-order group  $\mathbb{G}$  (Ristretto255 [53]), where the common-reference string consists of 3 group elements and each commitment is a single group element of 32 bytes, which achieves  $\lambda = 128$  bits of security under the discrete logarithm [54, 55] assumption.

### A.3 Security of TFDE as a Threshold PKE

**Definition 12 (Threshold PKE).** Let  $\mathcal{P} = \{P_1, \dots, P_N\}$  denote a set of parties and  $\mathbb{S}$  denote a class of efficient access structures over  $\mathcal{P}$ . A threshold public-key encryption scheme TPKE for message space  $\mathcal{M}$  and access structure class  $\mathbb{S}$  consists of the following polynomial-time algorithms:

- **Setup**( $1^\lambda, \mathbb{A}$ )  $\rightarrow (pp, ek, sk_1, \dots, sk_N)$ : Takes as input the security parameter  $\lambda$  and an access structure  $\mathbb{A} \in \mathbb{S}$ , and outputs public parameters  $pp$ , an encryption key  $ek$ , and key shares  $sk_1, \dots, sk_N$ .
- **Enc**( $ek, m$ )  $\rightarrow ct$ : Takes as input the encryption key  $ek$  and a message  $m \in \mathcal{M}$ , and outputs a ciphertext  $ct$ .
- **PartDec**( $pp, sk_i, ct$ )  $\rightarrow m_i$ : Takes as input the public parameters  $pp$ , a key share  $sk_i$ , and a ciphertext  $ct$ , and outputs a partial decryption share  $m_i$ .

- $\text{PartVerify}(pp, ct, m_i) \rightarrow \{0, 1\}$ : Takes as input the public parameters  $pp$ , a ciphertext  $ct$ , and a partial decryption share  $m_i$ , and outputs accept (1) or reject (0).
- $\text{Combine}(pp, \mathcal{B}) \rightarrow m'$ : Takes as input the public parameters  $pp$  and a set of partial decryption shares  $\mathcal{B} = \{m_i\}_{i \in S}$ , and outputs a message  $m'$ .

**Definition 13 (Compactness).** A threshold PKE scheme for access structure class  $\mathbb{S}$  satisfies compactness if there exist polynomials  $\text{poly}_1(\cdot)$  and  $\text{poly}_2(\cdot)$  such that for all security parameters  $\lambda$  and access structures  $\mathbb{A} \in \mathbb{S}$ , whenever  $(pp, ek, sk_1, \dots, sk_N) \leftarrow \text{Setup}(1^\lambda, \mathbb{A})$  and  $ct \leftarrow \text{Enc}(ek, m)$  for any message  $m$ , it holds that  $|ct| \leq \text{poly}_1(\lambda)$  and  $|ek| \leq \text{poly}_2(\lambda)$ .

**Definition 14 (Decryption Correctness).** A threshold PKE scheme for access structure class  $\mathbb{S}$  satisfies decryption correctness if for all security parameters  $\lambda$ , access structures  $\mathbb{A} \in \mathbb{S}$ , authorized sets  $S \in \mathbb{A}$ , and messages  $m \in \mathcal{M}$ , the following probability is overwhelming:

$$\Pr \left[ \begin{array}{l} (pp, ek, sk_1, \dots, sk_N) \leftarrow \text{Setup}(1^\lambda, \mathbb{A}); \quad ct \leftarrow \text{Enc}(ek, m); \\ \{m_i \leftarrow \text{PartDec}(pp, sk_i, ct)\}_{i \in S}; \quad m' \leftarrow \text{Combine}(pp, \{m_i\}_{i \in S}); \\ \text{return } (m' = m) \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

**Definition 15 (Partial Verification Correctness).** A threshold PKE scheme for access structure class  $\mathbb{S}$  satisfies partial verification correctness if for all security parameters  $\lambda$ , access structures  $\mathbb{A} \in \mathbb{S}$ , authorized sets  $S \in \mathbb{A}$ , parties  $i \in S$ , and messages  $m \in \mathcal{M}$ , the following probability is overwhelming:

$$\Pr \left[ \begin{array}{l} (pp, ek, sk_1, \dots, sk_N) \leftarrow \text{Setup}(1^\lambda, \mathbb{A}); \quad ct \leftarrow \text{Enc}(ek, m); \\ m_i \leftarrow \text{PartDec}(pp, sk_i, ct); \quad b \leftarrow \text{PartVerify}(pp, ct, m_i); \\ \text{return } (b = 1) \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

**Definition 16 (Security).** A threshold PKE scheme for access structure class  $\mathbb{S}$  satisfies security (IND-CCA) if for any efficient adversary  $\mathcal{A}$ , the advantage

$$\text{Adv}_{\mathcal{A}}^{\text{TPKE}}(\lambda) = |\Pr[\text{Expt}_{\mathcal{A},0}(1^\lambda) = 1] - \Pr[\text{Expt}_{\mathcal{A},1}(1^\lambda) = 1]|$$

is negligible in  $\lambda$ , where experiment  $\text{Expt}_{\mathcal{A},b}(1^\lambda)$  is defined as Figure 10.

**Theorem 5.** TFDE protocol (Definition 5) satisfies compactness (Definition 13) if the underlying FDE has polynomial-size messages and UT has compact partial evaluations.

The ciphertext of TFDE is the transcript  $\text{tr}^* = (\mu_1, \mu_2, \mu_3)$ , which is produced by  $\text{UT.Combine}$  and is identical in size to the FDE transcript regardless of  $n_S, n_R$ , since  $\text{UT.Combine}$  outputs the evaluation  $C(x)$  whose size depends only on the circuit output, not the number of parties.

**Theorem 6.** TFDE (Definition 5) satisfies decryption correctness (Definition 14) if FDE satisfies correctness (Definition 4) and UT satisfies evaluation correctness (Definition 3).

When all parties are honest and  $|A_S| \geq t_S$ ,  $|A_R| \geq t_R$ , each  $\text{UT.Combine}$  invocation faithfully recovers  $P_1(s, m)$ ,  $P_2(r, \mu_1)$ ,  $P_3(s, m, \mu_1, \mu_2)$ , and  $\text{FDE.Dec}(r, \mu_1, \mu_2, \mu_3)$  by evaluation correctness of UT, and the latter outputs  $m$  by correctness of FDE.

**Theorem 7.** The TFDE protocol (Definition 5) satisfies partial verification correctness (Definition 15) if UT satisfies verification correctness (Definition 3).

| $\text{Exp}_{\mathcal{A},0}^{\text{TPKE}}(1^\lambda)$                                                                                                                                                                                                                                                                                                                               | $\text{Exp}_{\mathcal{A},1}^{\text{TPKE}}(1^\lambda)$                                                                                                                                                                                                                                                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1: $b \leftarrow 0$                                                                                                                                                                                                                                                                                                                                                                 | 1: $b \leftarrow 1$                                                                                                                                                                                                                                                                                                                                                                 |
| 2: $\mathbb{A} \leftarrow \mathcal{A}(1^\lambda)$                                                                                                                                                                                                                                                                                                                                   | 2: $\mathbb{A} \leftarrow \mathcal{A}(1^\lambda)$                                                                                                                                                                                                                                                                                                                                   |
| 3: $(\text{pp}, \text{ek}, \text{sk}_1, \dots, \text{sk}_N) \leftarrow \text{Setup}(1^\lambda, \mathbb{A})$                                                                                                                                                                                                                                                                         | 3: $(\text{pp}, \text{ek}, \text{sk}_1, \dots, \text{sk}_N) \leftarrow \text{Setup}(1^\lambda, \mathbb{A})$                                                                                                                                                                                                                                                                         |
| 4: $S^* \leftarrow \mathcal{A}(\text{pp}, \text{ek})$ // where $S^* \notin \mathbb{A}$ , maximal invalid                                                                                                                                                                                                                                                                            | 4: $S^* \leftarrow \mathcal{A}(\text{pp}, \text{ek})$                                                                                                                                                                                                                                                                                                                               |
| 5: $b' \leftarrow \mathcal{A}^{\text{OPartDec}}(\{\text{ski}\}_{i \in S^*})$                                                                                                                                                                                                                                                                                                        | 5: $b' \leftarrow \mathcal{A}^{\text{OPartDec}}(\{\text{ski}\}_{i \in S^*})$                                                                                                                                                                                                                                                                                                        |
| 6: $(m_0^*, m_1^*) \leftarrow \mathcal{A}^{\text{OPartDec}}()$                                                                                                                                                                                                                                                                                                                      | 6: $(m_0^*, m_1^*) \leftarrow \mathcal{A}^{\text{OPartDec}}()$                                                                                                                                                                                                                                                                                                                      |
| 7: $\text{ct}_b^* \leftarrow \text{Enc}(\text{ek}, m_b^*)$                                                                                                                                                                                                                                                                                                                          | 7: $\text{ct}_b^* \leftarrow \text{Enc}(\text{ek}, m_b^*)$                                                                                                                                                                                                                                                                                                                          |
| 8: $b' \leftarrow \mathcal{A}^{\text{OPartDec}}(\text{ct}_b^*)$                                                                                                                                                                                                                                                                                                                     | 8: $b' \leftarrow \mathcal{A}^{\text{OPartDec}}(\text{ct}_b^*)$                                                                                                                                                                                                                                                                                                                     |
| 9: <b>return</b> $b'$                                                                                                                                                                                                                                                                                                                                                               | 9: <b>return</b> $b'$                                                                                                                                                                                                                                                                                                                                                               |
| <div style="border: 1px dashed black; padding: 5px; margin-top: 10px;"> <math>\text{OPartDec}(\text{ct}, i)</math> // <math>i \in [N] \setminus S^*</math><br/> <b>if</b> <math>\text{ct} = \text{ct}_b^*</math> <b>then return</b> <math>\perp</math><br/> <math>m_i \leftarrow \text{PartDec}(\text{pp}, \text{ski}, \text{ct})</math><br/> <b>return</b> <math>m_i</math> </div> | <div style="border: 1px dashed black; padding: 5px; margin-top: 10px;"> <math>\text{OPartDec}(\text{ct}, i)</math> // <math>i \in [N] \setminus S^*</math><br/> <b>if</b> <math>\text{ct} = \text{ct}_b^*</math> <b>then return</b> <math>\perp</math><br/> <math>m_i \leftarrow \text{PartDec}(\text{pp}, \text{ski}, \text{ct})</math><br/> <b>return</b> <math>m_i</math> </div> |

Fig. 10: IND-CCA Security Experiments for Threshold PKE

Every honestly computed partial evaluation  $y_i = \text{UT.Eval}(\text{pp}, s_i, \mathbb{C})$  passes the value of  $\text{UT.Verify}(\text{pp}, y_i, \mathbb{C}) = 1$  with overwhelming probability by verification correctness of UT.

**Theorem 8.** *The TFDE protocol (Definition 5) satisfies IND-CPA security against any PPT adversary controlling up to  $t_S - 1$  sender parties and up to  $t_R - 1$  receiver parties, if FDE is semantically secure and UT satisfies security (Definition 3).*

The adversary's view consists of  $\text{pp}_S$ ,  $\text{pp}_R$ ,  $\text{tr}^*$ , and at most  $t_\sigma - 1$  shares per side; the shares reveal nothing about  $x_S = (s || m)$  or  $x_R = r$  by security of UT, and the transcript  $\text{tr}^*$  hides  $m$  by semantic security of FDE (which follows from bideniability [16]).

#### A.4 Proof of Theorem 1

*Proof.* We show that no PPT adversary  $\mathcal{A}$  can distinguish the real world ( $b=0$ ) from the fake world ( $b=1$ ) in Definition 6 with more than negligible advantage, where  $H_0$  is the real world and  $H_4$  is the fake world.

**H<sub>0</sub>.** The challenger runs an honest execution of TFDE with plaintext  $m_0$ : it samples master randomness  $(s^*, r^*)$ , runs **ShareGen** to produce shares  $\{s_i^S\}_{i \in [n_S]}$ ,  $\{s_j^R\}_{j \in [n_R]}$ , executes the three threshold rounds to produce transcript  $\text{tr}^* = (\mu_1, \mu_2, \mu_3)$ , and upon receiving the coercion sets  $(C_S, C_R)$  from  $\mathcal{A}$  (with  $|C_S| < t_S$ ,  $|C_R| < t_R$ ), returns the *true* views  $\text{view}_{C_S} \cup \text{view}_{C_R}$ , each consisting of the party's share  $s_i^S$ , all partial evaluations  $\{y_i^\ell\}_\ell$ , commitment openings  $\{(c_i^\ell, d_i^\ell)\}_\ell$ , and NCE randomness  $\{r_{i,\ell}^G, r_{i,\ell}^E\}$ .

**H<sub>1</sub>.** Identical to  $H_0$  except: the challenger runs the honest execution with plaintext  $m_1$  (instead of  $m_0$ ) to obtain  $\text{tr}^*$ , then computes  $s' \leftarrow \text{FDE.SFake}(s^*, m_1, m_0, \text{tr}^*)$  and  $r' \leftarrow \text{FDE.RFake}(\hat{m} = m_0, \text{tr}^*; \rho)$ . However, it still uses the *original* shares and all threshold-layer state; only the master-level randomness that would be revealed is replaced by  $(s', r')$ .

We claim  $|\Pr[\mathcal{A}(H_0)=1] - \Pr[\mathcal{A}(H_1)=1]| \leq \text{negl}(\lambda)$ . Suppose  $\mathcal{A}$  distinguishes  $H_0$  from  $H_1$  with non-negligible advantage  $\epsilon$ . We construct an adversary  $\mathcal{B}$  against the bideniability of FDE (Definition 4).  $\mathcal{B}$  receives a challenge tuple  $(s_\beta, r_\beta, \text{tr}^*)$  from the FDE bideniability challenger, where  $\beta=0$  corresponds to real randomness for  $m_0$  and  $\beta=1$  corresponds to fake randomness consistent with  $m_0$  while the execution used  $m_1$ .  $\mathcal{B}$  uses  $(s_\beta, r_\beta)$  as the master randomness, runs **UT.Setup** with  $x_S = (s_\beta \| m_0)$  and  $x_R = r_\beta$  to generate shares, executes the threshold-layer machinery, and presents the resulting views to  $\mathcal{A}$ . If  $\mathcal{A}$  outputs  $b'$ ,  $\mathcal{B}$  outputs  $b'$ . The advantage of  $\mathcal{B}$  equals  $\epsilon$ , contradicting FDE bideniability.

Note that the transcript  $\text{tr}^* = (\mu_1, \mu_2, \mu_3)$  is identical in both hybrids since  $\mu_\ell = \text{UT.Combine}(\text{pp}, \{y_i^\ell\}_{i \in A})$  recovers  $P_\ell(\cdot)$  by UT evaluation correctness, and FDE.SFake/FDE.RFake produce randomness consistent with the *same* transcript.

**H<sub>2</sub>**. Identical to **H<sub>1</sub>** except: for each coerced sender  $i \in C_S$ , instead of providing the real share  $s_i^S$  derived from  $x_S = (s^* \| m_1)$ , the challenger computes  $\hat{x}_S = (s' \| m_0)$  and samples  $\tilde{s}_i^S \leftarrow \text{ShareSim}_S(C_S, \hat{x}_S)$ , providing  $\tilde{s}_i^S$  in place of  $s_i^S$ . Analogously, for each coerced receiver  $j \in C_R$ , it computes  $\hat{x}_R = r'$  and samples  $\tilde{s}_j^R \leftarrow \text{ShareSim}_R(C_R, \hat{x}_R)$ .

We claim  $|\Pr[\mathcal{A}(H_1)=1] - \Pr[\mathcal{A}(H_2)=1]| \leq \text{negl}(\lambda)$ . This follows from the security property of UT (Definition 3). Since  $|C_S| < t_S$  and  $|C_R| < t_R$ , the sets  $C_S$  and  $C_R$  are unauthorized. By the UT security guarantee, the joint distribution of shares  $\{s_i^\sigma\}_{i \in C_\sigma}$  for any unauthorized set is simulatable: there exists a PPT simulator  $\mathcal{S}$  such that for any unauthorized  $S^* \notin \mathbb{A}$ ,

$$\{s_i\}_{i \in S^*} \approx_c \mathcal{S}(\text{pp}, S^*, \hat{x})$$

for any target secret  $\hat{x}$ . Concretely, the  $\{0,1\}$ -LSSS underlying the thresholdizer guarantees that any  $t-1$  shares are information-theoretically independent of the shared secret, so re-sampling shares consistent with  $\hat{x}_S$  (resp.  $\hat{x}_R$ ) for a sub-threshold coalition produces an identically distributed restricted view. Any distinguisher between **H<sub>1</sub>** and **H<sub>2</sub>** directly yields a distinguisher against UT security.

**H<sub>3</sub>**. Identical to **H<sub>2</sub>** except: for each coerced party  $i \in C_S$  (resp.  $j \in C_R$ ) and each round  $\ell$ , instead of providing the real commitment opening  $(c_i^\ell, d_i^\ell)$  generated via  $\text{EQ.Commit}(1^\lambda, \text{crs}_{\text{EQ}}, (s_i^S, y_i^\ell))$ , the challenger uses the equivocation mechanism: it runs  $(\widehat{\text{crs}}, \tilde{c}_i^\ell, \text{st}_i^\ell) \leftarrow \text{EQ.Equiv}_1(1^\lambda)$  and then  $\tilde{d}_i^\ell \leftarrow \text{EQ.Equiv}_2(1^\lambda, \text{st}_i^\ell, (\tilde{s}_i^S, \hat{y}_i^\ell))$ , where  $\hat{y}_i^\ell = \text{UT.Eval}(\text{pp}_S, \tilde{s}_i^S, C_\ell)$  is the partial evaluation under the fake share.

We claim  $|\Pr[\mathcal{A}(H_2)=1] - \Pr[\mathcal{A}(H_3)=1]| \leq |C_S \cup C_R| \cdot L \cdot \text{negl}(\lambda)$ , where  $L \in \{2, 3\}$  is the number of rounds in which each party participates. For each coerced party and each round, we invoke the equivocability property of EQ (Definition 1). The equivocability guarantee states that the distribution  $(\widehat{\text{crs}}, \tilde{c}, \tilde{d})$  produced by  $\text{EQ.Equiv}_1$  and  $\text{EQ.Equiv}_2$  is computationally indistinguishable from an honest  $(\text{crs}, c, d)$ . We apply a standard hybrid argument over the at most  $(|C_S| + |C_R|) \cdot L$  commitment instances: in sub-hybrid **H<sub>3,k</sub>**, the first  $k$  commitments (in some fixed ordering) use equivocated openings and the remaining use real openings. Each transition incurs negligible distinguishing advantage by EQ equivocability combined with EQ CRS indistinguishability (which guarantees that the simulated CRS  $\widehat{\text{crs}}$  is indistinguishable from the real  $\text{crs}$ ). By a union bound over polynomially many transitions, the total advantage is negligible.

**H<sub>4</sub>**. Identical to **H<sub>3</sub>** except: for every NCE channel  $(\text{pk}_{i,j}, \text{sk}_{i,j})$  involving a coerced party (either  $i \in C_S \cup C_R$  or  $j \in C_S \cup C_R$ ), instead of providing the real randomness  $(r_{i,j}^G, r_{i,j}^E)$ , the challenger uses NCE simulatability: it retrieves the simulation state  $\text{st}_{i,j}^{\text{NCE}}$

from the NCE setup phase (where  $(\text{pk}_{i,j}, \text{ct}_{i,j}, \text{st}_{i,j}^{\text{NCE}}) \leftarrow \text{NCE.Sim}_1(1^\lambda)$ ) and computes  $(\tilde{r}_{i,j}^G, \tilde{r}_{i,j}^E) \leftarrow \text{NCE.Sim}_2(\hat{y}_i^\ell, \text{st}_{i,j}^{\text{NCE}})$ .

We claim  $|\Pr[\mathcal{A}(\text{H}_3) = 1] - \Pr[\mathcal{A}(\text{H}_4) = 1]| \leq \text{negl}(\lambda)$ . We invoke the simulatability property of NCE (Definition 2). The simulatability guarantee states that for any PPT adversary, the real distribution  $\{(\text{pk}, \text{ct}, r_G, r_E)\}$  is computationally indistinguishable from the simulated distribution  $\{(\text{pk}, \text{ct}, \tilde{r}_G, \tilde{r}_E)\}$ . We apply a hybrid argument over all NCE channels involving coerced parties. There are at most  $(|C_S| + |C_R|) \cdot (n_S + n_R) \cdot L$  such channels (polynomial in  $\lambda$ ). Each transition incurs negligible advantage by NCE simulatability; the total advantage is negligible by a union bound.

$\text{H}_4$  is exactly the fake world of Definition 6: the views presented to  $\mathcal{A}$  consist of fake shares  $\tilde{s}_i^\sigma$  consistent with the cover message  $m_0$ , fake partial evaluations  $\hat{y}_i^\ell$  recomputed under  $\tilde{s}_i^\sigma$ , equivocated commitment openings  $\tilde{d}_i^\ell$ , and equivocated NCE randomness—exactly the output of *ThreshSFake* and *ThreshRFake*.

By collecting the hybrid transitions we will have:

$$\begin{aligned} & |\Pr[\mathcal{A}(\text{H}_0) = 1] - \Pr[\mathcal{A}(\text{H}_4) = 1]| \\ & \leq \sum_{k=0}^3 |\Pr[\mathcal{A}(\text{H}_k) = 1] - \Pr[\mathcal{A}(\text{H}_{k+1}) = 1]| \\ & \leq \text{Adv}_{\text{FDE}}^{\text{biD}}(\lambda) + \text{Adv}_{\text{UT}}^{\text{sec}}(\lambda) + (|C_S| + |C_R|) \cdot L \cdot \text{Adv}_{\text{EQ}}^{\text{equiv}}(\lambda) \\ & \quad + (|C_S| + |C_R|)(n_S + n_R)L \cdot \text{Adv}_{\text{NCE}}^{\text{sim}}(\lambda) \\ & \leq \text{negl}(\lambda), \end{aligned}$$

since each term is negligible by assumption.  $\square$

## A.5 Proof of Theorem 2 (Threshold Off-the-Record Deniability)

*Proof.* We must show that no PPT adversary  $\mathcal{A}$  can distinguish among the three cases  $b \in \{0, 1, 2\}$  of Definition 7 with advantage exceeding  $\text{negl}(\lambda)$ . It suffices to show pairwise indistinguishability:  $\text{H}^{(0)} \approx_c \text{H}^{(1)} \approx_c \text{H}^{(2)}$ . We show  $\text{H}^{(0)} \approx_c \text{H}^{(1)}$ ; the argument for  $\text{H}^{(1)} \approx_c \text{H}^{(2)}$  is symmetric, and  $\text{H}^{(0)} \approx_c \text{H}^{(2)}$  follows by transitivity. In case  $b = 0$ : the real plaintext is  $m_0$ ; the sender's views are truthful; the receiver's views are faked toward  $m_1$ . In case  $b = 1$ : the real plaintext is  $m_1$ ; the sender's views are faked toward  $m_0$ ; the receiver's views are truthful.

$\text{G}_0 = \text{H}^{(0)}$ . Execute with  $m_0$ . Give  $\mathcal{A}$  true sender views for  $C_S$  and fake receiver views for  $C_R$  (via *ThreshRFake*(...,  $m_1, \text{tr}^*; \rho$ )).

$\text{G}_1$  (*Equivocate receiver-side threshold layer*). For each coerced receiver  $j \in C_R$ : replace real share  $s_j^R$  with  $\tilde{s}_j^R \leftarrow \text{ShareSim}_R(C_R, r')$  where  $r' \leftarrow \text{FDE.RFake}(m_1, \text{tr}^*; \rho)$ ; recompute partial evaluations; equivocate commitments via *EQ.Equiv*<sub>1,2</sub>; equivocate NCE channels via *NCE.Sim*<sub>2</sub>. By the same argument as in Theorem 1 (Hybrids  $\text{H}_2$ – $\text{H}_4$  applied only to the receiver side),  $\text{G}_0 \approx_c \text{G}_1$  with negligible distinguishing advantage bounded by  $\text{Adv}_{\text{UT}}^{\text{sec}}(\lambda) + |C_R| \cdot L \cdot \text{Adv}_{\text{EQ}}^{\text{equiv}}(\lambda) + |C_R| \cdot (n_S + n_R) \cdot L \cdot \text{Adv}_{\text{NCE}}^{\text{sim}}(\lambda)$ .

*Hybrid  $\text{G}_2$  (FDE-level OTR switch).* Switch the underlying execution plaintext from  $m_0$  to  $m_1$ , and simultaneously switch sender views from truthful to faked (toward  $m_0$ ) and receiver views from faked to truthful. Concretely: the transcript is now  $\text{tr}^* = \text{tr}(s^*, r^*, m_1)$ ; the sender views for  $C_S$  are generated via *FDE.SFake*( $s^*, m_1, m_0, \text{tr}^*$ ); the receiver views for  $C_R$  use real  $(r^*, m_1)$ .

We claim  $G_1 \approx_c G_2$ . At this point,  $G_1$  corresponds to case (i) of the FDE OTR game (sender truth, receiver fakes, plaintext  $m_0$ ) and  $G_2$  corresponds to case (ii) (sender fakes, receiver truth, plaintext  $m_1$ ). We construct  $\mathcal{B}$  against FDE OTR as follows:  $\mathcal{B}$  receives the challenge  $(s_\beta, r_\beta, \text{tr}^*)$  from the FDE OTR challenger (where  $\beta \in \{0,1\}$  selects between cases (i) and (ii)), wraps the threshold layer around  $(s_\beta, r_\beta)$  by running  $\text{UT.Setup}$  with the appropriate secret, and presents the resulting views to  $\mathcal{A}$ . The threshold-layer equivocation for the coerced side (whichever is faking) is performed as in  $G_1$  or  $G_3$  below. Any distinguishing advantage of  $\mathcal{A}$  translates directly to  $\mathcal{B}$ 's advantage against FDE OTR.

*Hybrid  $G_3$  (Equivocate sender-side threshold layer).* For each coerced sender  $i \in C_S$ : replace real share  $s_i^S$  with  $\tilde{s}_i^S \leftarrow \text{ShareSim}_S(C_S, \hat{x}_S)$  where  $\hat{x}_S = (s' \| m_0)$  and  $s' \leftarrow \text{FDE.SFake}(s^*, m_1, m_0, \text{tr}^*)$ ; recompute partial evaluations; equivocate commitments and NCE channels. This is  $H^{(1)}$ .

$G_2 \approx_c G_3$  by the identical argument as  $G_0 \approx_c G_1$ , applied to the sender side. Case  $b = 2$  uses real plaintext  $m_2$ , with both sender and receiver views faked (sender toward  $m_0$ , receiver toward  $m_1$ , possibly inconsistent). The argument is analogous: first equivocate both sides' threshold layers (shares, commitments, NCE), then invoke the FDE OTR three-way indistinguishability (cases (ii)  $\approx_c$  (iii)) to switch from  $(m_1, \text{sender fakes, receiver truth})$  to  $(m_2, \text{both fake})$ .

$$\begin{aligned} & \left| \Pr[\mathcal{A} \text{ wins in } H^{(b)}] - \frac{1}{3} \right| \\ & \leq \text{Adv}_{\text{FDE}}^{\text{OTR}}(\lambda) + 2 \cdot \text{Adv}_{\text{UT}}^{\text{sec}}(\lambda) + 2(|C_S| + |C_R|) \cdot L \cdot \text{Adv}_{\text{EQ}}^{\text{equiv}}(\lambda) \\ & \quad + 2(|C_S| + |C_R|)(n_S + n_R)L \cdot \text{Adv}_{\text{NCE}}^{\text{sim}}(\lambda) \leq \text{negl}(\lambda). \end{aligned}$$

The factor of 2 accounts for equivocating both the sender and receiver threshold layers across the pairwise transitions.  $\square$

## A.6 Proof of Theorem 3

*Proof.* We must verify that  $\text{ThreshRFake}$  (Figure 7) satisfies the restricted syntax of Definition 8: its inputs are  $(\text{CRS}_{\text{TFDE}}, \text{msk}, \text{pp}_R, C_R, \hat{m}, \mu_1, \mu_2, \mu_3; \rho)$  and it does *not* require the real receiver master randomness  $r$ , the real plaintext  $m$ , or the real receiver shares  $\{s_j^R\}_{j \in C_R}$ .

We verify each step of  $\text{ThreshRFake}$ :

**FDE-level receiver faking.**  $r' \leftarrow \text{FDE.RFake}(\hat{m}, \mu_1, \mu_2, \mu_3; \rho)$ . By the public receiver deniability of FDE [16],  $\text{FDE.RFake}$  has syntax  $\text{RFake}(\hat{m}, \text{tr}; \rho)$  and does *not* take the real receiver randomness  $r^*$  or the real plaintext  $m$  as input. This is an explicit design property of the FDE construction in [16]: the receiver's faking algorithm uses only the public transcript and the target fake plaintext together with fresh random coins  $\rho$ .

**Fake share construction.**  $\hat{x}_R \leftarrow r'$ ;  $\tilde{s}_j^R \leftarrow \text{ShareSim}_R(C_R, \hat{x}_R)$  for  $j \in C_R$ . The share simulator  $\text{ShareSim}_R$  takes as input only the coerced set  $C_R$  and the target secret  $\hat{x}_R$ . It does *not* require the real shares  $\{s_j^R\}$ . This follows from the information-theoretic property of the  $\{0,1\}$ -LSSS underlying UT: for any unauthorized set  $C_R$  with  $|C_R| < t_R$ , one can sample shares consistent with *any* target secret without knowledge of the original shares, because the conditional distribution  $\{s_j^R\}_{j \in C_R} | x_R$  depends only on  $x_R$  (not on the shares of parties outside  $C_R$ ). Formally, the simulator samples from the marginal distribution of the sharing of  $\hat{x}_R$  restricted to coordinates in  $C_R$ .



**Fake partial evaluations.**  $\hat{y}_j^\ell \leftarrow \text{UT.Eval}(\text{pp}_R, \tilde{s}_j^R, \mathbb{C}_\ell)$  for  $j \in C_R$ ,  $\ell \in \{2, \text{dec}\}$ . This uses only the public parameters  $\text{pp}_R$ , the fake shares  $\tilde{s}_j^R$  (produced in Step 2), and the circuit  $\mathbb{C}_\ell$  (determined by the public transcript). No real receiver secret is required.

**Commitment equivocation.**  $(\widehat{\text{crs}}_j, \hat{c}_j^\ell, \text{st}_j^\ell) \leftarrow \text{EQ.Equiv}_1(1^\lambda)$ ;  $\tilde{d}_j^\ell \leftarrow \text{EQ.Equiv}_2(1^\lambda, \text{st}_j^\ell, (\tilde{s}_j^R, \hat{y}_j^\ell))$ . The equivocation algorithms  $\text{EQ.Equiv}_1$  and  $\text{EQ.Equiv}_2$  use only the security parameter and the target opening value. They operate via the equivocation trapdoor  $\text{td}_{\text{EQ}}$  embedded in  $\text{msk}$ , not via any receiver-specific secret.

**NCE channel equivocation.**  $(\tilde{r}_{j,\cdot}^G, \tilde{r}_{j,\cdot}^E) \leftarrow \text{NCE.Sim}_2(\hat{y}_j^\ell, \text{st}_{j,\cdot}^{\text{NCE}})$ . The NCE simulation algorithm  $\text{NCE.Sim}_2$  takes the target plaintext  $\hat{y}_j^\ell$  and the simulation state  $\text{st}_{j,\cdot}^{\text{NCE}}$  (which is part of  $\text{msk}$ , generated during setup by  $\text{NCE.Sim}_1$ ). It does *not* require the real NCE randomness or decryption keys specific to any receiver.

As a result, the complete input set of  $\text{ThreshRFake}$  is:

1.  $\text{CRS}_{\text{TFDE}}$ : public;
2.  $\text{msk} = (\text{td}_{\text{EQ}}, \{\text{st}_{i,j}^{\text{NCE}}\}_{i,j})$ : system-level secret state, *not* receiver-specific;
3.  $\text{pp}_R$ : public parameters of the receiver-side thresholdizer;
4.  $C_R$ : the set of coerced receivers (indices);
5.  $\hat{m}$ : the fake plaintext;
6.  $(\mu_1, \mu_2, \mu_3)$ : the public transcript;
7.  $\rho$ : fresh random coins.

None of the items  $r$ ,  $m$ , or  $\{s_j^R\}_{j \in C_R}$  appear in this list. Therefore, any entity possessing  $\text{msk}$ , not necessarily any of the coerced receivers, can produce valid fake receiver views.  $\square$

## A.7 Proof of Theorem 5

*Proof.* We verify Definition 13. In TFDE, the “ciphertext” presented to the receiver coalition is the transcript  $\text{tr}^* = (\mu_1, \mu_2, \mu_3)$ , where each  $\mu_\ell = \text{UT.Combine}(\text{pp}_\sigma, \{y_i^\ell\}_{i \in A_\sigma})$  is the output of the UT combine algorithm applied to the partial evaluations of the circuit  $\mathbb{C}_\ell$ . By the compactness property of UT (Definition 3), each partial evaluation  $y_i^\ell$  has size  $|y_i^\ell| \leq \text{poly}(\lambda, d, N)$ , independent of the secret  $x_\sigma$ . The output of  $\text{UT.Combine}$  is the circuit evaluation  $\mathbb{C}_\ell(x_\sigma)$ , which equals the FDE round function output  $P_\ell(\cdot)$ . By assumption, the FDE protocol messages  $P_1(s, m)$ ,  $P_2(r, \mu_1)$ ,  $P_3(s, m, \mu_1, \mu_2)$  each have size  $\text{poly}(\lambda)$  (since FDE operates on polynomial-length messages under subexponentially secure iO and OWFs [16]). Therefore  $|\text{tr}^*| = |\mu_1| + |\mu_2| + |\mu_3| \leq 3 \cdot \text{poly}(\lambda) = \text{poly}_1(\lambda)$ , which is independent of  $n_S$  and  $n_R$ .

The encryption key in TFDE corresponds to the compound CRS  $\text{CRS}_{\text{TFDE}} = (\text{CRS}_{\text{FDE}}, \text{crs}_{\text{EQ}}, \{\text{pk}_{i,j}, \text{sk}_{i,j}\}_{i,j})$  together with  $\text{pp}_S, \text{pp}_R$ . The CRS components have size  $\text{poly}(\lambda)$ ; the NCE key pairs number  $O((n_S + n_R)^2)$ , each of size  $\text{poly}(\lambda)$ ; the public parameters  $\text{pp}_\sigma$  have size  $\text{poly}(\lambda, d, N)$  by UT compactness. Thus  $|\text{ek}| \leq \text{poly}(\lambda, d, n_S, n_R) = \text{poly}_2(\lambda)$ , since  $d, n_S, n_R$  are all polynomial in  $\lambda$ .  $\square$

## A.8 Proof of Theorem 6

*Proof.* We verify Definition 14. Suppose all parties are honest and  $|A_S| \geq t_S$ ,  $|A_R| \geq t_R$ . We trace the protocol execution.

ShareGen runs  $(\text{pp}_S, s_1^S, \dots, s_{n_S}^S) \leftarrow \text{UT.Setup}(1^\lambda, 1^d, (\mathbb{S}_S, n_S), x_S)$  with  $x_S = (s \| m)$ , and  $(\text{pp}_R, s_1^R, \dots, s_{n_R}^R) \leftarrow \text{UT.Setup}(1^\lambda, 1^d, (\mathbb{S}_R, n_R), x_R)$  with  $x_R = r$ .

**Round 1.** Each sender  $i \in A_S$  computes  $y_i^1 \leftarrow \text{UT.Eval}(\text{pp}_S, s_i^S, \mathbb{C}_1)$ , where  $\mathbb{C}_1$  is defined so that  $\mathbb{C}_1(x_S) = P_1(s, m)$ . Since  $A_S \in \mathbb{S}_S$  (authorized), by evaluation correctness of UT:

$$\mu_1 = \text{UT.Combine}(\text{pp}_S, \{y_i^1\}_{i \in A_S}) = \mathbb{C}_1(x_S) = P_1(s, m)$$

with overwhelming probability. The verification step  $\text{UT.Verify}(\text{pp}_S, y_i^1, \mathbb{C}_1) = 1$  holds for all honest parties by verification correctness of UT, so no honest party aborts.

**Round 2.** Each receiver  $j \in A_R$  computes  $y_j^2 \leftarrow \text{UT.Eval}(\text{pp}_R, s_j^R, \mathbb{C}_2)$  where  $\mathbb{C}_2(x_R) = P_2(r, \mu_1)$  with  $\mu_1$  hardcoded. By evaluation correctness:

$$\mu_2 = \text{UT.Combine}(\text{pp}_R, \{y_j^2\}_{j \in A_R}) = \mathbb{C}_2(x_R) = P_2(r, \mu_1).$$

**Round 3.** We will have:

$$\mu_3 = \text{UT.Combine}(\text{pp}_S, \{y_i^3\}_{i \in A_S}) = \mathbb{C}_3(x_S) = P_3(s, m, \mu_1, \mu_2).$$

The circuit  $\mathbb{C}_{\text{dec}}$  is defined so that  $\mathbb{C}_{\text{dec}}(x_R) = \text{FDE.Dec}(r, \mu_1, \mu_2, \mu_3)$  with the transcript hardcoded. By evaluation correctness:

$$m' = \text{UT.Combine}(\text{pp}_R, \{y_j^{\text{dec}}\}_{j \in A_R}) = \mathbb{C}_{\text{dec}}(x_R) = \text{FDE.Dec}(r, \mu_1, \mu_2, \mu_3).$$

By the correctness of FDE (Definition 4)  $\text{FDE.Dec}(r, \text{tr}(s, r, m)) = m$  except with negligible probability over uniform  $s, r$ . Since each  $\text{UT.Combine}$  invocation succeeds with overwhelming probability (by evaluation correctness of UT), and  $\text{FDE.Dec}$  succeeds with overwhelming probability (by correctness of FDE), a union bound over the four  $\text{UT.Combine}$  calls and the single  $\text{FDE.Dec}$  call yields:

$$\Pr[\text{ThreshDec}(\text{CRS}, \text{pp}_R, \{s_j^R\}_{j \in A_R}, \mu_1, \mu_2, \mu_3) = m] \geq 1 - 5 \cdot \text{negl}(\lambda) = 1 - \text{negl}(\lambda). \quad \square$$

## A.9 Proof of Theorem 7

*Proof.* We verify Definition 15. In TFDE, partial decryption corresponds to the computation of partial evaluations  $y_i^\ell = \text{UT.Eval}(\text{pp}_\sigma, s_i^\sigma, \mathbb{C}_\ell)$  in each round  $\ell$ , and partial verification corresponds to  $\text{UT.Verify}(\text{pp}_\sigma, y_i^\ell, \mathbb{C}_\ell)$ . Fix any round  $\ell \in \{1, 2, 3, \text{dec}\}$ , any honest party  $i$  in the active set  $A_\sigma$ , and any circuit  $\mathbb{C}_\ell$  determined by the protocol. Since party  $i$  is honest, it computes  $y_i^\ell \leftarrow \text{UT.Eval}(\text{pp}_\sigma, s_i^\sigma, \mathbb{C}_\ell)$  faithfully.

By the verification correctness property of UT (Definition 3): for any honestly computed partial evaluation  $y_i = \text{UT.Eval}(\text{pp}, s_i, C)$ ,

$$\Pr[\text{UT.Verify}(\text{pp}, y_i, C) = 1] \geq 1 - \text{negl}(\lambda).$$

This directly applies to each partial evaluation in TFDE. Since each party produces at most  $L \leq 4$  partial evaluations (rounds 1, 3 for senders; rounds 2, dec for receivers), and each passes verification with overwhelming probability, a union bound over all  $|A_\sigma| \cdot L$  verification checks yields:

$$\Pr[\forall i \in A_\sigma, \forall \ell: \text{UT.Verify}(\text{pp}_\sigma, y_i^\ell, \mathbb{C}_\ell) = 1] \geq 1 - |A_\sigma| \cdot L \cdot \text{negl}(\lambda) = 1 - \text{negl}(\lambda),$$

since  $|A_\sigma| \leq N = \text{poly}(\lambda)$ . Moreover, the robustness property of UT ensures that no PPT adversary, even given all shares  $s_1, \dots, s_N$ , can produce a fake partial evaluation  $y_i^* \neq \text{UT.Eval}(\text{pp}, s_i, C)$  that passes  $\text{UT.Verify}$ , except with negligible probability. This guarantees that the verification step in each round of TFDE rejects any maliciously crafted partial evaluations with overwhelming probability, preventing corrupted parties from injecting invalid partial computations.  $\square$

### A.10 Proof of Theorem 8

*Proof.* We show that no PPT adversary  $\mathcal{A}$  controlling at most  $t_S - 1$  sender parties and at most  $t_R - 1$  receiver parties can distinguish encryptions of  $m_0$  from encryptions of  $m_1$  with more than negligible advantage. The proof proceeds through three hybrids.

$H_0$ . The challenger executes TFDE with plaintext  $m_0$ . The adversary  $\mathcal{A}$  receives  $\text{CRS}_{\text{TFDE}}$ ,  $\text{pp}_S$ ,  $\text{pp}_R$ , the transcript  $\text{tr}^* = (\mu_1, \mu_2, \mu_3)$ , and the shares of corrupted parties  $\{s_i^S\}_{i \in S_S^*}, \{s_j^R\}_{j \in S_R^*}$  where  $|S_S^*| < t_S$  and  $|S_R^*| < t_R$ .

$H_1$ . Identical to  $H_0$  except: the corrupted parties' shares are replaced by simulated shares. Specifically, for the sender side, instead of providing real shares  $\{s_i^S\}_{i \in S_S^*}$  derived from  $x_S = (s \| m_0)$ , the challenger runs the UT simulator:

$$\{\tilde{s}_i^S\}_{i \in S_S^*} \leftarrow \mathcal{S}_{\text{UT}}(\text{pp}_S, \mathbb{C}_1, \mathbb{C}_1(x_S), S_S^*).$$

Note that the simulator receives only the public parameters, the circuit, and the circuit output  $\mathbb{C}_1(x_S) = \mu_1$  (not  $x_S$  itself). Analogously for the receiver side.

We claim  $H_0 \approx_c H_1$ . This follows directly from the security property of UT (Definition 3): since  $S_S^* \notin \mathbb{S}_S$  and  $S_R^* \notin \mathbb{S}_R$  (sub-threshold, hence unauthorized), no PPT adversary can distinguish real shares and partial evaluations from simulated ones. Formally, by definition:

$$(\text{pp}_\sigma, \{s_i^\sigma\}_{i \in S_\sigma^*}, \{y_i^\ell\}_{i \in S_\sigma^*}) \approx_c (\text{pp}_\sigma, \mathcal{S}_{\text{UT}}(\text{pp}_\sigma, \mathbb{C}_\ell, \mathbb{C}_\ell(x_\sigma), S_\sigma^*)).$$

The distinguishing advantage is  $\leq \text{Adv}_{\text{UT}}^{\text{sec}}(\lambda) = \text{negl}(\lambda)$ .

$H_2$ . Identical to  $H_1$  except the execution uses plaintext  $m_1$  instead of  $m_0$ . Observe that in  $H_1$ , the adversary's view consists of:

1. The transcript  $\text{tr}^* = (\mu_1, \mu_2, \mu_3)$ , which equals  $(P_1(s, m_b), P_2(r, \mu_1), P_3(s, m_b, \mu_1, \mu_2))$ ;
2. Simulated shares that depend only on  $\text{pp}_\sigma$  and the circuit outputs  $\{\mu_1, \mu_2, \mu_3, m'\}$  (where  $m' = \text{FDE.Dec}(r, \text{tr}^*)$ );
3. Public parameters  $\text{pp}_S, \text{pp}_R$ .

After the UT simulation step, the adversary's information about the plaintext comes *solely* through the transcript  $\text{tr}^*$  (since the simulated shares and partial evaluations depend only on the circuit outputs, not on  $x_S$  or  $x_R$  directly).

We claim  $H_1 \approx_c H_2$ . We construct an adversary  $\mathcal{B}$  against the semantic security of FDE.  $\mathcal{B}$  receives  $(\text{CRS}_{\text{FDE}}, \text{tr}^*)$  from the FDE semantic security challenger, where  $\text{tr}^*$  is either an encryption of  $m_0$  or  $m_1$ .  $\mathcal{B}$  embeds this into TFDE by constructing the remaining CRS components ( $\text{crs}_{\text{EQ}}$ , NCE keys), generating simulated shares via UT simulation (which requires only  $\text{tr}^*$  and public parameters), and presenting the resulting view to  $\mathcal{A}$ .

Formally,  $\mathcal{B}$  sets up the UT instances with circuits  $\mathbb{C}_1, \mathbb{C}_2, \mathbb{C}_3$  such that their outputs match the components of  $\text{tr}^*$ . The UT simulator  $\mathcal{S}_{\text{UT}}$  can produce simulated shares given only the circuit outputs (which are the components of  $\text{tr}^*$ ). Thus,  $\mathcal{B}$  can construct the complete adversarial view without knowing the plaintext.

Any advantage  $\epsilon$  of  $\mathcal{A}$  in distinguishing  $H_1$  from  $H_2$  translates to an advantage  $\epsilon$  of  $\mathcal{B}$  against FDE semantic security. Since FDE is semantically secure (which follows from bideniability [16]: bideniability implies that the transcript hides the plaintext, as otherwise a distinguisher between real and fake randomness would exist), we have  $\epsilon \leq \text{Adv}_{\text{FDE}}^{\text{sem}}(\lambda) = \text{negl}(\lambda)$ .

$$\begin{aligned}
\text{Adv}_{\text{TFDE}, \mathcal{A}}^{\text{IND-CPA}}(\lambda) &= |\Pr[\mathcal{A}(H_0) = 1] - \Pr[\mathcal{A}(H_2) = 1]| \\
&\leq |\Pr[\mathcal{A}(H_0) = 1] - \Pr[\mathcal{A}(H_1) = 1]| + |\Pr[\mathcal{A}(H_1) = 1] - \Pr[\mathcal{A}(H_2) = 1]| \\
&\leq 2 \cdot \text{Adv}_{\text{UT}}^{\text{sec}}(\lambda) + \text{Adv}_{\text{FDE}}^{\text{sem}}(\lambda) \\
&\leq \text{negl}(\lambda),
\end{aligned}$$

where the factor of 2 for UT accounts for simulation on both sender and receiver sides.  $\square$

## References

1. Shafi Goldwasser, Saleet Klein, and Daniel Wichs. The edited truth. In *Theory of Cryptography: 15th International Conference, TCC 2017, Baltimore, MD, USA, November 12-15, 2017, Proceedings, Part I*, page 305–340, Berlin, Heidelberg, 2017. Springer-Verlag.
2. Dana Dachman-Soled. On minimal assumptions for sender-deniable public key encryption. In *Proceedings of the 17th International Conference on Public-Key Cryptography — PKC 2014 - Volume 8383*, page 574–591, Berlin, Heidelberg, 2014. Springer-Verlag.
3. Jesper Buus Nielsen. Separating random oracle proofs from complexity theoretic proofs: The non-committing encryption case. In Moti Yung, editor, *Advances in Cryptology — CRYPTO 2002*, volume 2442 of *LNCS*. Springer, 2002.
4. Daniel Apon, Xinyu Fan, and Feng-Hao Liu. Deniable attribute based encryption for branching programs from LWE. In Martin Hirt and Adam Smith, editors, *Theory of Cryptography*, volume 9986 of *LNCS*. Springer, 2016.
5. Ran Canetti, Cynthia Dwork, Moni Naor, and Rafail Ostrovsky. Deniable encryption. In Burton S. Kaliski, editor, *Advances in Cryptology — CRYPTO ’97*, volume 1294 of *LNCS*. Springer, 1997.
6. Angelo De Caro, Vincenzo Iovino, and Adam O’Neill. Deniable functional encryption. In *Public Key Cryptography (1)*, pages 196–222. Springer, 2016.
7. Daniel Hugenroth, Alberto Sonnino, Sam Cutler, and Alastair R. Beresford. Sloth: Key stretching and deniable encryption using secure elements on smartphones. *IACR Cryptol. ePrint Arch.*, 2023:1792, 2024.
8. Katriel Cohn-Gordon, Cas Cremers, Benjamin Dowling, Luke Garratt, and Douglas Stebila. A formal security analysis of the signal messaging protocol. In *2017 IEEE European Symposium on Security and Privacy (EuroSecP)*, pages 451–466, 2017.
9. Niusen Chen, Bo Chen, and Weisong Shi. An application-layer plausibly deniable encryption system for wearable devices. *Discover Internet of Things*, 4(1), August 2024.
10. Niusen Chen, Bo Chen, and Weisong Shi. Mobiwear: a plausibly deniable encryption system for wearable mobile devices. In *EAI International Conference on Applied Cryptography in Computer and Communications*, pages 138–154. Springer, 2021.
11. GitHub - zorggomat/FractalCryptGUI: Free cross-platform deniable encryption cryptoarchiver — github.com. <https://github.com/zorggomat/{F}ractal{C}rypt{G}{U}{I}>. [Accessed 13-04-2026].
12. VeraCrypt - Free Open source disk encryption with strong security for the Paranoid — veracrypt.io. <https://veracrypt.io/en/Home.html>. [Accessed 16-12-2025].
13. Elia Anzuoni and Tommaso Gagliardoni. Shufflecake: Plausible deniability for multiple hidden filesystems on linux. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS ’23*, page 3033–3047, New York, NY, USA, 2023. Association for Computing Machinery.
14. Amit Sahai and Brent Waters. How to use indistinguishability obfuscation: deniable encryption, and more. In *Proceedings of the forty-sixth annual ACM symposium on Theory of computing*, pages 475–484, 2014.

15. Adam O'Neill, Chris Peikert, and Brent Waters. Bi-deniable public-key encryption. In *Annual Cryptology Conference*, pages 525–542. Springer, 2011.
16. Ran Canetti, Sunoo Park, and Oxana Poburinnaya. Fully deniable interactive encryption. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020*, volume 12170 of *LNCS*. Springer, Cham, 2020.
17. Mona Soleymani, Navid Abapour, Elham Taghizadeh, Safieh Siadat, and Rasoul Karkehabadi. Fuzzy rule-based trust management model for the security of cloud computing. *Mathematical Problems in Engineering*, 2021(1):6629449, 2021.
18. Jinglong Luo, Yehong Zhang, Zhuo Zhang, Shiyu Liu, Ye Dong, Haoran Li, Yue Yu, Hui Wang, Xun Zhou, and Zenglin Xu. Cryptography-based privacy-preserving large language models: a lifecycle survey of frameworks, methods, and future directions. *Artificial Intelligence Review*, 59(2), December 2025.
19. Yi Zhang, Kun Tian, Yunfan Lu, Fengxia Liu, Cheng Li, Zixian Gong, Zhe Hu, Jia Li, and Qun Xu. Repairable threshold paillier encryption scheme for federated learning. *Soft Computing*, 29(7):3659–3664, April 2025.
20. Multi-Party Threshold Cryptography | CSRC | CSRC — [csrc.nist.gov. https://csrc.nist.gov/projects/threshold-cryptography](https://csrc.nist.gov/projects/threshold-cryptography). [Accessed 13-04-2026].
21. Rene Peralta. NIST Internal or Interagency Report (NISTIR) 8214C, NIST First Call for Multi-Party Threshold Schemes — [csrc.nist.gov. https://csrc.nist.gov/pubs/ir/8214/c/final](https://csrc.nist.gov/pubs/ir/8214/c/final). [Accessed 13-04-2026].
22. Kiarash Sedghighadikolaei and Attila Altay Yavuz. A survey of threshold signatures: Nist standards, post-quantum cryptography, exotic techniques, and real-world applications. *ACM Comput. Surv.*, October 2025. Just Accepted.
23. Ehasn Ebrahimi and Anshu Yadav. Strongly secure universal thresholdizer. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024*, pages 207–239. Springer Nature Singapore, 2025.
24. Josh Benaloh, Michael Naehrig, Olivier Pereira, and Dan S. Wallach. ElectionGuard: a cryptographic toolkit to enable verifiable elections. In *USENIX Security*, pages 5485–5502, Philadelphia, PA, August 2024. USENIX Association.
25. ElectionGuard - Official — [electionguard.vote. https://www.electionguard.vote/spec/](https://www.electionguard.vote/spec/). [Accessed 20-07-2024].
26. Manage quorum authentication (M of N access control) using CloudHSM CLI - AWS CloudHSM — [docs.aws.amazon.com. https://docs.aws.amazon.com/cloudhsm/latest/userguide/quorum-auth-chsm-cli.html](https://docs.aws.amazon.com/cloudhsm/latest/userguide/quorum-auth-chsm-cli.html). [Accessed 13-04-2026].
27. DigiCert documentation — [docs.digicert.com. https://docs.digicert.com/?lang=en](https://docs.digicert.com/?lang=en). [Accessed 13-04-2026].
28. Alexander Ziller, Andrew Trask, Antonio Lopardo, Benjamin Szymkow, Bobby Wagner, Emma Bluemke, Jean-Mickael Nounahon, Jonathan Passerat-Palmbach, Kritika Prakash, Nick Rose, Théo Ryffel, Zarreen Naowal Reza, and Georgios Kaissis. *PySyft: A Library for Easy Federated Learning*, page 111–139. Springer International Publishing, 2021.
29. Dan Boneh, Rosario Gennaro, Steven Goldfeder, and Sam Kim. A lattice-based universal thresholdizer for cryptographic systems. Cryptology ePrint Archive, Paper 2017/251, 2017.
30. Aayush Jain, Peter M. R. Rasmussen, and Amit Sahai. Threshold fully homomorphic encryption. Cryptology ePrint Archive, Paper 2017/257, 2017.
31. Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018*, pages 565–596. Springer International Publishing, 2018.
32. Lior Rotem and Gil Segev. *Non-malleable Vector Commitments via Local Equivocability*, page 415–446. Springer International Publishing, 2021.
33. Zvika Brakerski, Pedro Branco, Nico Döttling, Sanjam Garg, and Giulio Malavolta. *Constant Ciphertext-Rate Non-committing Encryption from Standard Assumptions*, page 58–87. Springer International Publishing, 2020.

34. Rikke Bendlin, Jesper Buus Nielsen, Peter Sebastian Nordholt, and Claudio Orlandi. Lower and upper bounds for deniable public-key encryption. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 125–142. Springer, 2011.
35. Shaoquan Jiang, Yeow Meng Chee, San Ling, Huaxiong Wang, and Chaoping Xing. A new framework for deniable secure key exchange. *Information and Computation*, 285:104866, 2022.
36. Ran Canetti, Ivan Damgård, Sebastian Kolby, Divya Ravi, and Sophia Yakubov. Deniable secret sharing. In *Theory of Cryptography: 23rd International Conference, TCC 2025, Aarhus, Denmark, December 1–5, 2025, Proceedings, Part II*, page 399–427, Berlin, Heidelberg, 2025. Springer-Verlag.
37. Ivan Damgård and Jesper Buus Nielsen. Improved non-committing encryption schemes based on a general complexity assumption. In Mihir Bellare, editor, *Advances in Cryptology — CRYPTO 2000*, volume 1880 of *LNCS*. Springer, 2000.
38. Atsushi Waseda and Ryo Nojima. On limitation of plausible deniability. In Kazumaro Aoki and Akira Kanaoka, editors, *Advances in Information and Computer Security. IWSEC 2020*, volume 12231 of *LNCS*. Springer, Cham, 2020.
39. S. Garg et al. Threshold encryption with silent setup. In *CRYPTO*, pages 352–386. Springer, 2024.
40. B. Waters and D.J. Wu. Silent threshold cryptography from pairings: Expressive policies in the plain model. Cryptology ePrint Archive, Paper 2025/1547, 2025.
41. M. Hall-Andersen, M. Simkin, and B. Wagner. Silent threshold encryption with one-shot adaptive security. Cryptology ePrint Archive, Paper 2025/1384, 2025.
42. J. Bormet et al. BEAST-MEV: Batched threshold encryption with silent setup for MEV prevention. Cryptology ePrint Archive, Paper 2025/1419, 2025.
43. Johan Håstad, Russell Impagliazzo, Leonid A. Levin, and Michael Luby. A pseudorandom generator from any one-way function. volume 28, pages 1364–1396. SIAM, 1999.
44. Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. In *Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing (STOC) 1985 (and published in full in SIAM J. Comput.)*, volume 18 of *SIAM Journal on Computing*, pages 186–208. SIAM, 1989.
45. Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *J. ACM*, 56(6), September 2009.
46. Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In *Advances in Cryptology – EUROCRYPT 2010*, volume 6110 of *Lecture Notes in Computer Science*, pages 1–23. Springer, 2010.
47. Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013*, volume 8042 of *LNCS*. Springer, 2013.
48. Alfredo De Santis, Silvio Micali, and Giuseppe Persiano. Non-interactive zero-knowledge with preprocessing. In Shafi Goldwasser, editor, *Advances in Cryptology — CRYPTO’88*, volume 403 of *LNCS*. Springer, New York, NY, 1990.
49. Manuel Blum. Coin flipping by telephone a protocol for solving impossible problems. *SIGACT News*, 15(1):23–27, January 1983.
50. Zvika Brakerski, Pedro Branco, Nico Döttling, Sanjam Garg, and Giulio Malavolta. Constant ciphertext-rate non-committing encryption from standard assumptions. In *Theory of Cryptography – TCC 2020*, volume 12550 of *Lecture Notes in Computer Science*, pages 58–87. Springer, 2020.
51. Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In *Proceedings of the 37th Annual ACM Symposium on Theory of Computing (STOC 2005)*, pages 84–93. ACM, 2005.

- 52. Venkatesan Guruswami and Madhu Sudan. List decoding algorithms for certain concatenated codes. In *Proceedings of the 32nd Annual ACM Symposium on Theory of Computing (STOC 2000)*, pages 181–190. ACM, 2000.
- 53. Mike Hamburg. Decaf: Eliminating cofactors through point compression. In *Advances in Cryptology – CRYPTO 2015*, page 705–723, Berlin, Heidelberg, 2022. Springer-Verlag.
- 54. John M. Pollard. Monte carlo methods for index computation (mod  $p$ ). *Mathematics of Computation*, 32(143):918–924, 1978.
- 55. National Institute of Standards and Technology. FIPS PUB 186-5: Digital Signature Standard (DSS). Technical report, NIST, 2023.